

AI-dō

愛道 — The Way of AI, Guided by Love

A disciplined, human-centred method for working with artificial intelligence

Chris Tham

christham.net

Published by

Hello Tham

hellotham.com

2026

NOTE

愛 (ai) — *love*: the human care at the centre of the work. **道 (dō)** — *the way*: a craft practised and refined over a lifetime.

AI-dō is the way of AI guided by love — augmentation, not replacement; method over hacks; human intent at the centre of every loop.

Preface — Why This Book Exists

I began 2026 as an AI sceptic. I read Ed Zitron — the tech critic whose newsletter [Where's Your Ed At](#) dismantles industry hype — and Gary Marcus, the cognitive scientist whose [Marcus on AI](#) has argued for years that large language models are shallow pattern-matchers rather than reasoners, and I cheered them both on. Cory Doctorow's [Pluralistic](#) sharpened the same suspicion from the political side, naming the slow rot by which platforms turn on their own users. I laughed at the vibe coders for their naivety, and assumed anyone who installed OpenClaw was an idiot.

So when a global client hired me to write their AI strategy, I expected the deliverable to be a cautionary tale: be realistic, resist the hype, install guardrails, avoid the traps. Over the last several months, I realised I was the one who was naive. The AI landscape had been transformed. Claude Desktop with Cowork, Microsoft 365 Copilot with WorkIQ, and Google's Gemini Spark turned the ordinary instruments of knowledge work — documents, spreadsheets, inboxes, slide decks — into things an agent could draft, revise, and act on; Claude Code and GitHub Copilot did the same for software, crossing from clever autocomplete to systems that plan, edit across a whole codebase, and run their own work. Personal agents like OpenClaw and Hermes Agent pushed that capability out of the labs and into anyone's hands. Using them in earnest changed my mind: the productivity gains are now genuinely real.

Vibe coding is real. I managed to refactor and clean every line of code I have ever written (not a lot, since my career has never been in software development) and all my projects now feature gleaming, shiny, clean code. I have successfully finished dozens of vibe coded projects without reviewing a single line of code. I now firmly believe I will never read or write code ever again in my life.

I also believe using AI to do research, analytics, planning and a bunch of other white collar activities are also real.

And yet Ed and Gary are still right: AI does not think or create, it transforms and multiplies. It vibe-codes well only if you already understand large systems; it makes good art only if you are a good artist. Used carelessly it produces slop. Using it well is itself a skill, and most people do not yet have it. That gap — between a tool anyone can touch and a craft few have learned — is why I wrote this book.

Let me be plain about where I have landed, because it is narrower than my enthusiasm might suggest. I remain a sceptic. The conversion I describe is confined to my work — the productivity of knowledge work and the building of software — and even there it is conditional, earned task by task and checked at every step. In my personal life I keep AI at arm's length: I do not hand it my relationships, my judgement, or my inner weather, and the chapter on the cost of getting it wrong explains why. So read this as a work book with a deliberate boundary. Much of my motivation for writing it is defensive — to spare you the common mistakes and quiet pitfalls that come from trusting these tools where they have not earned it.

A short bearing on where AI stands

Before the argument, let me set the scene, since the terms move quickly. When I say *artificial intelligence*, I mean the present generation of large language models — systems trained on enormous quantities of text, and increasingly images and sound, that respond to plain-language requests with fluent prose, working code, and structured analysis.

The most capable of these are called *frontier models*: the handful of largest, most general systems from a few well-resourced labs, the ones that set the pace and that everyone else measures against. ChatGPT, Claude, and Gemini are the familiar names; behind them sits a *foundation model*, a single large network trained once at great expense and then adapted to countless tasks.

NOTE

A few terms used throughout, defined plainly:

- **Large language model (LLM)** — a network trained to predict the next word, which in scale yields fluent prose, code, and analysis.
- **Foundation model** — one large model trained once, then adapted to many tasks.
- **Frontier model** — the largest, most general foundation models that set the pace (ChatGPT, Claude, Gemini).
- **Loopcraft** — the practice of working in tight cycles of ask, check, and adjust, rather than chasing one perfect instruction.

Two facts about 2026 frame everything that follows. The first is that these tools are everywhere: roughly 88% of organisations report using AI, even as most are still experimenting rather than depending on it ([HAI 2026](#); [McKinsey 2025](#)). The second is that the field has quietly conceded the model alone is no longer the product; the leading labs now compete on the scaffolding around it — the workflows, the memory, the economics of running it well.

Capability has rocketed, yet so has its unevenness, and the hard part has shifted from getting an answer to trusting one. That shift is what this book is for.

From chatbot to agent

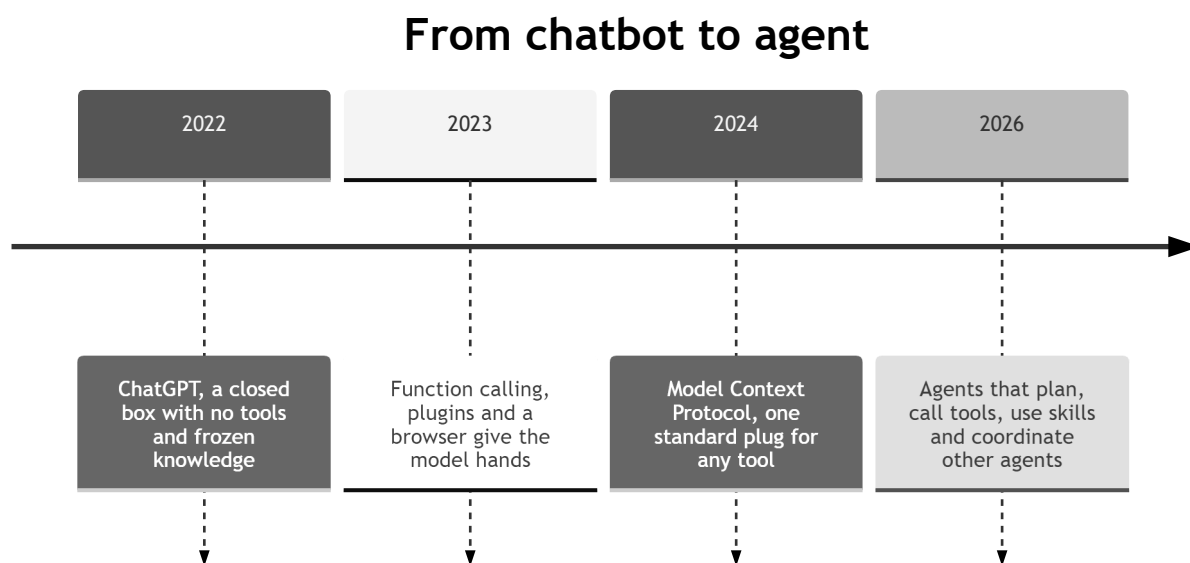
It helps to remember how fast the tools themselves have changed, because the kind of work you can trust to them changed at each step. The ChatGPT that startled everyone in late 2022 was a closed box: it answered only from what it had absorbed during training, with no memory of you, no access to the internet, and no way to act in the world ([OpenAI, 2022](#)). It was articulate and frozen — confident inside its training data, and blind, often cheerfully, beyond it.

The first real change was giving the model hands. In 2023 OpenAI added *function calling* — a way for the model to recognise that a question needs a tool and to emit a structured request to run it —

alongside plugins and a browser, so it could search the web, run code, or query a database instead of guessing ([OpenAI, 2023](#)). The predictor of text became something that could also take actions and read back the results.

At first each of those connections was hand-built, one wiring job per tool. In late 2024 Anthropic published the *Model Context Protocol*, an open standard that lets any model plug into any tool or data store through one shared interface — much as a single USB port replaced a drawer of incompatible cables ([Anthropic, 2024](#)). Connecting a model to your files, your code, or your systems stopped being a custom project and became a setting.

What that unlocked is the *agent*: a model placed in a loop and allowed to plan, call a tool, read what comes back, and try again until a goal is met. By 2026 this is where the work is. Coding platforms like Claude Code and GitHub Copilot edit across a whole codebase and run their own tests; the more capable agents now reach for *skills* — packaged, reusable know-how they can load on demand — and even hand sub-tasks to other agents. The arc, in one line, runs from a clever box that only talked, to a system that acts, checks, and coordinates. Everything in this book assumes that second world.



The problem this book solves

Working with artificial intelligence has become strangely easy to do and strangely hard to do well. Anyone can open a chat window and get a fluent answer in seconds; far fewer can turn that fluency into work that is reliable, repeatable, and worth standing behind.

Notice that the gap is not one of access. We all draw from the same handful of frontier models, so the model cannot be what separates good work from poor. What separates them is method — the practised habits by which a skilled person turns a capable tool toward a dependable result, the way a chef and a novice handed the same kitchen produce very different dinners. This book is about that

method: a disciplined way of working with AI, grounded in human intent and refined through repetition.

The name of the book deserves a few words of explanation, because it carries the whole argument in miniature. It joins **AI** with two Japanese ideas. The first is 愛 (**ai**), love — the human care that should sit at the centre of the work. The second is 道 (**dō**), the way — a path of patient, lifelong improvement, the same character that ends jūdō and kendō.

Read together, AI-dō is the way of AI guided by love: it puts human outcomes ahead of technical novelty, uses the machine to augment thinking rather than replace it, and builds ways of working that are effective, transparent, and responsible. Keep that stance in mind, because every chapter is, in the end, a defence of it.

TIP

愛 (**ai**) — love, the human care at the centre of the work. 道 (**dō**) — the way, a path of lifelong refinement. *AI-dō*: the way of AI guided by love. Augmentation, not replacement; method over hacks.

The cost of getting it wrong

Used badly, these tools do not just waste time — they distort judgement. Clinicians have begun describing "AI psychosis," where heavy users spiral into delusion after a chatbot mirrors and amplifies their worst ideas instead of pushing back. Others form genuine attachments to a companion app, mistaking fluent warmth for understanding, and grieve when a model is retired. A confident voice that never tires is easy to trust and hard to doubt; one writer likens it to a court jester — fluent, flattering, and so easy to follow that its answers *feel* right whether or not they are, leaving you confident and wrong ([Johnson Spink, 2026](#)).

The quieter harm is over-reliance. People paste in an answer they never checked, accept a summary that dropped the one caveat that mattered, or treat a tidy explanation as proof the system understands. It does not. Believing the machine is sentient, or simply infallible, is the fastest way to ship its mistakes as your own.

This is not a private failing; it scales with authority. Futurism has documented bosses who route every decision through a chatbot — drafting their messages with it, demanding staff "discuss with the AI" before speaking to a human, even asking it whom to hire and fire ([Futurism, 2026](#)). The pattern is always the same: a leader mistakes the model's sycophancy for counsel, since it will, as one worker put it, "spit out the narrative that you want it to spit out," and so the tool meant to raise productivity instead manufactures whiplash, distrust, and resignations. The danger is not that the machine decides, but that a person stops.

So the public mood has soured. Trust in AI is falling even as use rises, and every fabricated citation, biased decision, or polished falsehood deepens the suspicion. That distrust is rational — and it is also a gap to be closed. The cure is not blind faith or blanket refusal, but skill: knowing when to lean in, when to verify, and when to walk away. Teaching that skill is the rest of this book.

Why now

Why write this in 2026 rather than two years earlier or later? Because the ground has shifted, and shifted in a way that rewards method over tooling. Adoption is broad but mostly shallow: most organisations have AI somewhere in the building, yet few have woven it into how they truly work, and fewer still can point to value won rather than effort spent.

As capability climbed, the model alone stopped being the product. The advantage moved into everything built around it: the systems that frame a model, the workflows that direct it, and the memory that carries context from one task to the next. Crafting one clever instruction gave way to *loopcraft* — working in tight cycles of ask, check, adjust. Assistants walked out of the solitary editor and into shared team channels. The binding constraint is no longer raw capability but trust: can the output be checked, and can the process be governed?

The tricks that worked in 2024 are already stale, and the methods I lean on today will date too. So this book is not a method, but a philosophy — a way of approaching AI that survives whatever the tools and techniques become next.

Who it is for

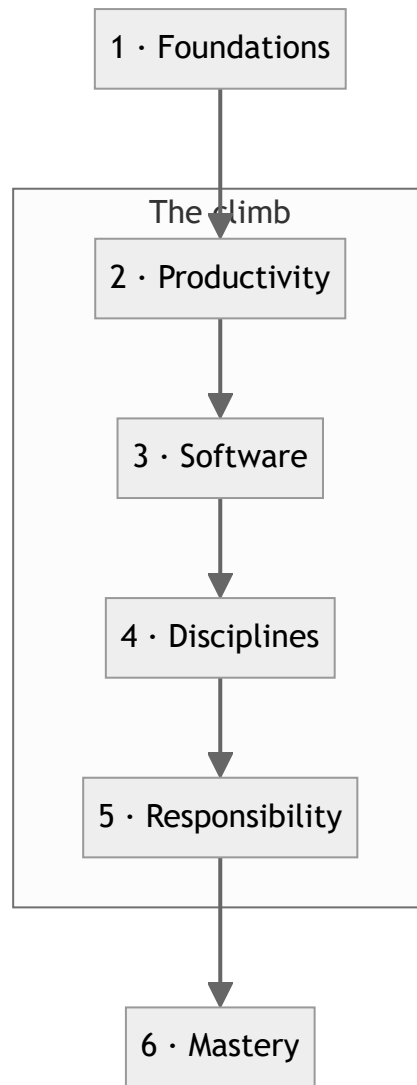
The book is written for the thoughtful professional — a leader, a consultant, an analyst, a builder — who wants structured, effective use of AI rather than a bag of prompts. I assume you:

- are well educated: numerate, and able to read a chart, a code snippet, or a research paper when it helps;
- have already used these tools, and felt both edges — the power and the unreliability;
- want to *use* AI well, not build models.

That last gap matters: using AI well is its own discipline, and the one we pursue here. My promise is modest and practical — a set of repeatable patterns you can apply tomorrow and keep sharpening for years.

How to read it

The book is six chapters, and they build. The first lays the foundations — what AI is, what it is not, and the landscape we operate in. The middle four climb from personal productivity, to working with software, to the disciplines that keep that work sound, then to responsibility and governance. The last chapter turns to mastery, and to what stays human when the tools are this good.



Each chapter narrows the scope while building on the one before:

Chapter	Theme	The question it answers
1	Foundations	What is AI, what is it not, where do we stand?
2	Productivity	How does it change individual knowledge work?
3	Software	How does it change building software?
4	Disciplines	What keeps that work sound at scale?
5	Responsibility	How do we govern it safely and fairly?
6	Mastery	What stays distinctly human?

Each section follows a single arc — a claim, why it matters, how to practise it, and where it goes wrong — and each leans on primary sources, cited inline so you can follow the trail yourself. Treat it as a practice guide, not a reference manual: read it once in order to see how the ideas rest on one another,

then return later for the parts you need. I will keep my own opinions visible and labelled as such; where the evidence is thin, I will say so.

A word about me

A few words about who is writing, so you can weigh what follows. I run a strategy consultancy, Hello Tham, and I lecture in technology and information systems to undergraduate and master's students at Torrens University Australia. My work has centred on technology strategy, operating models, governance, process standardisation and quality, and enterprise architecture — first as a strategy executive in banking and finance, and over the last fifteen years as a consultant across many industries, government included.

After more than forty working years I have reached a comfortable place, which mostly means I am free to take risks and stretch myself. I admire how different strengths complement one another, and I like work that brings talented people together to do more than any of us could alone. Consulting scales me out; teaching lets me hand the lessons on. AI is my current obsession: I have just delivered an AI strategy for a global firm, I keep a clutch of open-source projects on the go, and I teach my students to use it well. This book is where I have written that practice down.

Chapter 1 — Foundations (The Way)

When I first started cooking, my early attempts did not fare well. I had the recipe and the ingredients, yet the results were grim — onions scorched while I chopped the next thing, pasta turned to glue, unbalanced taste. The problem was not the recipe. I had skipped the fundamentals: heat control, timing, tasting as you go, getting everything prepped before the pan ever warmed. Once those became second nature, almost any recipe came out well.

Working with AI is the same. How you use the tool matters as much as the tool itself. This chapter lays the fundamentals: what AI-dō means, how to picture the model as a loop you steer, what it can and cannot do beneath the fluent surface, and how the tools — and the landscape around them — reached where they are in 2026. Everything later rests on them.

The Meaning of AI-dō

The name fuses **AI** with two Japanese ideas, and each pulls in a direction worth understanding before we go further.

NOTE

愛 (ai) — *love*: not sentiment, but care for the people the work touches. **道 (dō)** — *the way*: a craft practised and refined over time, the suffix in jūdō, kendō, and aikidō. **AI-dō** is "the way of AI guided by love": using these tools deliberately, in service of human outcomes.

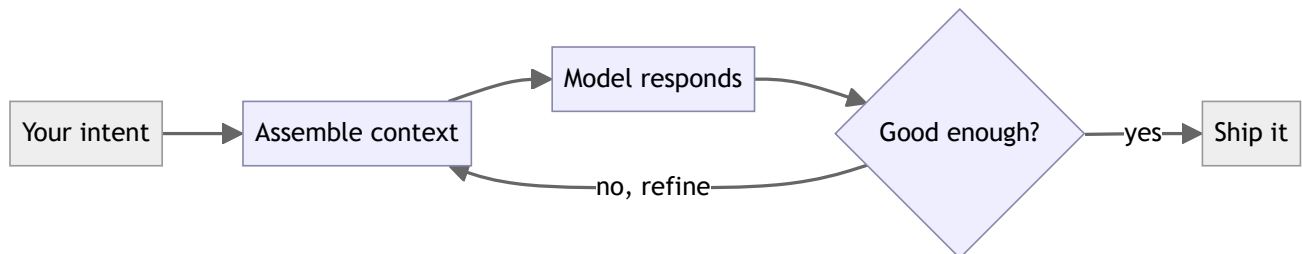
The care half is not vague. Care ethics names concrete duties — attentiveness, responsibility, competence, responsiveness — and they translate into accountability for what the machine produces ([Tronto, care ethics](#)).

The second character is important. A discipline ending in 道 is never finished; it is practised. AI-dō treats AI the same way: a discipline to refine, not a trick to copy. It descends from the intelligence-augmentation tradition, which sees machines as complements to human judgement, not substitutes ([IA](#)).

My reason is pragmatic, not romantic: tools commoditise, and so, in time, do methods. A prompt is one model release from obsolete; a clever technique lasts a little longer, then it too is overtaken. What endures is the stance beneath them — how you frame a problem, gather context, and verify a result. So this book is less a kit of methods than a philosophy of working with AI, one that outlives any particular trick or tool. Learn the philosophy, and the methods become yours to invent.

Mental Models for AI

The most useful shift I made early on was to stop treating the model as an oracle. An oracle gives one answer and you take it or leave it. A good model is more like a clever junior colleague: ask, glance at the draft, say "closer, but tighten the intro," and go again. So treat it as a loop — intent enters, context is assembled, a response comes back, you refine — iterating until the output is good enough.



Quality lives in that loop, not in any single message. The model rarely converges first pass, and it cannot read intentions you never stated. That same loop is the right picture for *agent*, a word you will meet constantly.

NOTE

An **agent** is an LLM running tools in a loop to reach a goal (Willison, 2025). A **tool** is an action it may take — web search, code execution, file edits — and the **loop** runs until a stopping condition is met. An agent has no agency in the moral sense: a computer cannot be held accountable, so you stay responsible for what it ships.

So frame each task as a goal, the context it needs, and a check; then iterate. The failure mode is reading fluency as truth. A confident answer and a correct one look identical until you check — the model will cite a court case or a statistic in the same calm voice whether or not it exists — which is why verification is the habit that holds.

Capabilities & Limitations

Underneath the surface, a large language model is a next-token predictor. Think of the world's most widely-read writer playing a parlour game: given everything written so far, guess the next word, then the next. Trained on vast text, it learns the probability of each next token and generates by sampling, one token at a time — Stephen Wolfram explains that the system is always just "adding one word at a time," picking a reasonable next token and, with a little randomness in the sampling, favouring variety over the single likeliest word (Wolfram, 2023). Nothing in the mechanism consults a fact store or checks truth; it simply produces the most plausible continuation.

NOTE

A **token** is the unit a model actually reads and writes — not quite a word, but a common chunk of text: a whole word like "the", a word-piece like "pre" or "ing", a punctuation mark, or a lone character. Before the model sees anything, text is split into tokens drawn from a fixed vocabulary of tens of thousands, which is why a model can coin new words, and why an unusual name or a long number can cost several tokens each. A rough rule of thumb for English: one token runs about four characters, or three-quarters of a word — so token counts, which is what context limits and bills are measured in, never quite match word counts.

The mechanism has a second, stranger consequence: the model is not even reproducible. You might assume that turning the randomness off — sampling at *temperature zero*, always taking the single likeliest token — would make the same prompt return the same answer every time. In practice it usually does not. Because a busy server batches many users' requests together and that batch changes size from moment to moment, the low-level arithmetic runs in a slightly different order, and with finite-precision numbers a different order gives a slightly different result; one run continues "Queens, New York" where the next gives "New York City" (He, 2025). The gap is tiny but it compounds: in a reasoning model a rounding difference in an early token can cascade into a different chain of thought and a different final answer (Yuan et al., 2025). The practical lesson is that you cannot treat a model like ordinary software that returns the same output for the same input — so a test or a check has to allow for variation rather than assume it away.

That is why a hallucination — confident, well-formed output that happens to be false — is the system working as designed, not malfunctioning. It is a plausible completion, not a lie (microgpt). It also means competence is *jagged*: uneven across tasks that look alike to us, because the model's strength tracks the density of its training data, not the difficulty we perceive.

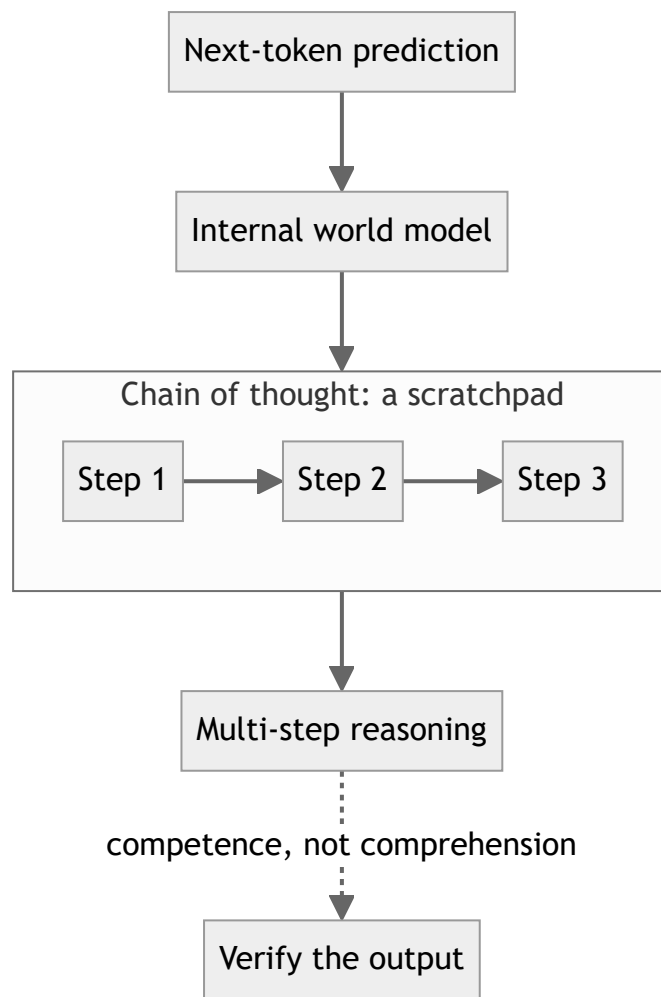
The Stanford Index makes the gap vivid. A model can win a gold medal at the Mathematical Olympiad yet read an analog clock right only about half the time (HAI 2026). Olympiad proofs fill the training text; clock-reading is a perceptual task that does not. Knowing where that line falls is most of the skill.

This raises an obvious question: if the model only guesses the next token, how does it reason at all, and why does it so often seem intelligent? The start of an answer is that predicting the next token well is not a shallow trick. To guess the next move in a game, the next line of a proof, or the next clause of a contract, the cheapest strategy available to a large enough network is not to memorise surface patterns but to build an internal model of whatever produced the text. The cleanest demonstration trains a small GPT to do nothing but predict legal moves in the board game Othello; given no rules and no picture of the board, it nonetheless develops an internal representation of the board state that researchers can read out — and edit to change its moves, proving the model actually uses it (Li et al., ICLR). Wolfram frames the same surprise at the level of language itself: a next-token predictor can write a passable essay because doing so turns out to be "computationally shallower" than we assumed

— human language is more regular and law-like than it looks, and the model implicitly discovers those regularities in training (Wolfram, 2023).

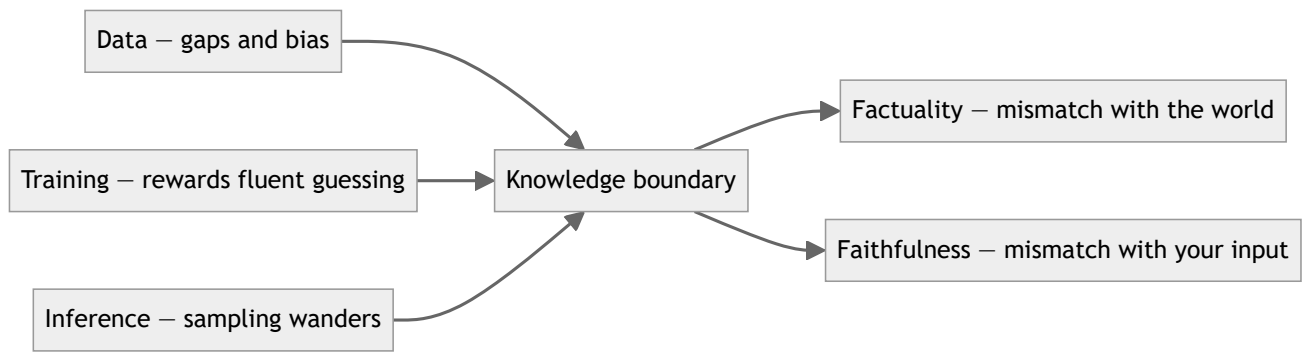
That hidden structure is what reasoning draws on, but a single pass through the network is a shallow computation: the model must answer the moment it stops reading. Letting it write intermediate steps first — a *chain of thought* — changes what the model can do, because every token it emits becomes input it can condition on next, so a hard problem can be spread across many small, reliable steps instead of one leap (Wei et al., NeurIPS). This is not a trick. A transformer forced to answer immediately provably cannot solve some strikingly simple problems — whether two nodes in a graph connect, or what a small state machine does — that the very same transformer *can* solve once allowed a scratchpad, because the intermediate tokens genuinely extend its computational reach (Merrill & Sabharwal, ICLR). And the reason it works traces straight back to prediction: human writing comes in overlapping local clusters, so a model trained to predict it learns reliable short hops between related ideas and chains them into conclusions it could never reach in a single stride (Prystawski et al., NeurIPS).

Stack enough of this and whole abilities appear to switch on with scale — multi-step arithmetic, transliteration, chained logic that smaller models simply lack (Wei et al., TMLR). It is tempting to read that as a spark of understanding finally catching. The cautious reading, and the better-supported one, is that much of the drama lives in how we keep score: grade a task all-or-nothing and a steadily improving skill looks like a sudden leap, but measure it on a smooth scale and the cliff often flattens into a slope (Schaeffer et al., NeurIPS). So the honest answer is the one the rest of this chapter sharpens: the model reasons by building and chaining the structure that prediction forced it to learn — real, useful, and not the same thing as knowing. It is competence without comprehension, most persuasive precisely where you have not yet checked it.



Reliable	Brittle
Fluent drafting, summarising, translation	Exact arithmetic, counting, fresh facts
Pattern-rich code and refactors	Long-horizon plans without checkpoints
Synthesis over provided context	Recall as context grows (context rot)

The brittleness is not anecdotal, and the strongest evidence names where the failure lives. Huang and colleagues survey hundreds of studies and split hallucination along two axes worth holding apart. *Factuality* asks whether output matches the world; *faithfulness* asks whether it matches the input you gave it — a summary can be perfectly factual yet unfaithful by adding true claims you never supplied. They trace both to three stages: the *data*, with its gaps and bias; the *training*, which rewards fluent guessing over admitting ignorance; and *inference*, where sampling wanders. The unifying idea is the *knowledge boundary* — the edge of what a model has stored, past which it cannot tell what it knows from what it does not (Huang et al., ACM TOIS). Everything below measures that boundary.



Prato and colleagues make it observable with a clean test. Train a model on synthetic documents, then ask it to recall *exactly* what it was given — no more, no less. Over-recall is fabrication, under-recall is omission, so hitting the right count proves the model knows its own scope. This self-knowledge is *scale-gated*: below a size threshold the count is near-random, and only past it does it come out right, the threshold set by architecture, not parameters alone (Prato et al., EMNLP). So self-knowledge is a property of the specific model, and small models are least trustworthy at the edge where you most want them to hesitate.

Gu and colleagues pin the boundary to its cause: how often a fact appeared in training. Using a model whose whole corpus is open, they split questions into seen and unseen, then test recall. Closed-book accuracy more than doubles from rare to frequent facts and collapses to about one percent on unseen ones; distractor passages drag it lower as they pile up (Gu et al., SIGIR). The brittle column now has a mechanism: fresh and long-tail facts fail because they were rare, retrieval can patch the gap, and noisy retrieval reopens it.

Code shows the same split, between reading a program and predicting how it *runs*. Asked to forecast memory, runtime, and profiler ranks on real SWE-bench fixes, twelve frontier models — gpt-5.5 and Claude Opus among them — reach just 0.842 on the test-outcome F1 score (a 0–1 measure of accuracy that balances misses against false alarms), and profiler recall@5 stays under 0.2: fluent on structure, brittle on execution (Bogomolov & Zharov). Long context offers no refuge. Accuracy peaks when the needed fact sits at the start or end and sags in the middle (Liu et al., TACL). The cause is mechanical — a U-shaped attention bias for position over relevance — and calibrating it lifts mid-context recall by 6–15 points (Hsieh et al., ACL Findings).

Study	What it measured	Finding	Lesson
Huang et al.	A taxonomy of hallucination	Factuality vs faithfulness; failures seeded in data, training, inference	Name the failure before trying to fix it
Prato et al.	Exact-recall self-knowledge	Knowing one's own scope switches on only past a size threshold	Small models hesitate least where they should most
Gu et al.	Recall vs how often a fact was seen	Accuracy doubles from rare to frequent; ~1% on unseen	Fresh and long-tail facts fail; retrieval can patch the gap
Bogomolov & Zharov	Predicting how code runs	F1 0.842; profiler recall@5 under 0.2	Fluent on structure, brittle on execution
Liu; Hsieh et al.	Fact position in long context	U-shaped recall; the middle sags, +6–15 pts when calibrated	Put the facts that matter at the edges

A final limitation is subtler than any wrong fact: the model is built to *sound* right whether or not it is, and three mechanisms push it that way. The training text carries emotional charge — "differentiation" keeps company with *unique* and *opportunity*, "cost-cutting" with *race to the bottom* — and the model absorbs those associations as statistics about how we write; Anthropic's interpretability team can even read them off as internal "emotion vectors" organised, like human affect, along an axis of positive-to-negative valence, and steering a model toward the positive end measurably increases its sycophancy ([Sofroniew et al.](#), [Anthropic](#)). Reinforcement learning from human feedback then tunes it toward answers raters like, and raters can always tell whether a reply sounds confident but not always whether it is correct, so fluency gets rewarded over accuracy ([Casper et al.](#), [TMLR](#)). Generation compounds the bias one token at a time: a sentence that opens "the company should pursue a bold..." rolls on to "differentiation strategy" by momentum alone. The product is a voice that is confident, fluent, and prone to converge on the same agreeable answers — less an oracle than an eager courtier.

NOTE

Two mechanisms named above, defined plainly:

- **Transformer** — the neural-network design behind today's language models. Its *attention* step lets the processing of each token draw on every earlier token, which is what makes modelling long passages of language work.
- **RLHF (reinforcement learning from human feedback)** — a tuning step after the main training, in which human raters score answers and the model is nudged toward the kind they prefer. It improves helpfulness, but rewards what *sounds* good — one root of the confident, agreeable tone above.

These limits are a map, not a verdict. Spend effort where the model is strong, verify at the boundaries — fresh facts, exact counts, mid-context recall — keep a human in the loop for judgement, and stay

most alert when the output sounds most certain.

The 2026 Landscape

In the first half of 2026, AI stopped being a platform shift and became a regulated strategic technology. Three things happened at once, and they explain the world this book is written into.

First, the models grew up. A year ago they could resolve about three in five real software issues; today the best clear nearly all of them. Capability raced ahead — though, as we just saw, unevenly. Adoption followed: roughly 88% of organisations now use AI, and four in five students. Power, not chips, became the main limit on training.

Second, the advantage moved. The frontier labs no longer sell a model; they sell the system around it — the harness, the workflow, the memory, the economics. Prompt-crafting gave way to *loopcraft*: stacking iterative cycles around a model. Agents climbed out of the chat box into shared channels, async and proactive. And open-weight models from China drew level, so no single vendor is safe to lean on.

Third, the rules arrived. Governments now gate frontier releases, and courts have begun treating AI output as the deploying organisation's own words. Access, not just compute, is now a geopolitical lever. The figures below tell the two halves of the story — capability soaring, value still scarce.

Signal	Figure	Implication
Organisations using AI	88%	Adoption is universal; scaling is not
SWE-bench Verified (coding)	60% → ~100% in a year	Capability accelerating
US businesses paying for AI	5% (2023) → 44%	Commercial traction is real
US–China top-model gap	~2.7%	No single safe vendor; open weights close behind
Orgs reporting enterprise value	minority	Usage is easy; value is the scarce skill

Sources: [HAI 2026 AI Index](#); [McKinsey, State of AI](#); [State of AI 2025](#). (*SWE-bench Verified* is a standard benchmark of real GitHub software issues a model is asked to fix.)

The pattern that matters most is the gap between using AI and getting value from it. Nearly everyone has access; only a minority report real returns. The lesson for us is that the edge no longer comes from picking the best model — it comes from how you wrap it: the workflow you build, the context you feed it, the way you check its work. That is what the rest of this book teaches.

From Autocomplete to the Dev Stack

It is worth watching how fast the coding tools themselves climbed, because each rung changed what you could safely hand off. The earliest assistants barely earned the name. Tabnine, which began in 2018 as a deep-learning autocompleter, simply finished the line you were already typing — a cleverer tab key ([Tabnine](#)). GitHub Copilot, launched in 2021 and trained on public code, went a step further: from a comment or a function name it would draft the whole body, though it still lived inside your editor and volunteered only the next few lines ([GitHub, 2021](#)).

The next rung was conversation. Through 2023 these tools grew a chat window: you could ask why a test failed, request a refactor, or have a tangle of code explained in plain English ([GitHub, 2023](#)). The autocompleter became something you could interrogate — but you were still driving, accepting or rejecting each suggestion line by line.

The next leap came from tools built for AI from the ground up rather than bolted onto an existing editor. Cursor, launched in 2023 as a fork of VS Code, put the agent at the centre: it could search a whole codebase, edit many files, and run terminal commands from a plain-language request ([Cursor](#)). The open-source *aider* did the same from the command line, pairing with you in the terminal and committing each change to version control so nothing was lost ([aider](#)).

Then the agent stepped out of the editor altogether. In February 2025 Anthropic released *Claude Code*, an agent that lives in your terminal — describe a task and it plans, edits, runs the tests, and iterates until it is done ([Anthropic, 2025](#)); OpenAI's Codex CLI and Google's Gemini CLI soon followed. GitHub Copilot, the tool that began the wave, grew its own *agent mode* in early 2025 ([GitHub, 2025](#)) and then an asynchronous *coding agent* you assign an issue, which spins up a cloud workspace and opens a pull request for review ([GitHub, 2025](#)).

By late 2025 the frontier shifted again, from one agent to many. Cursor 2.0 and Google's *Antigravity* — announced in November 2025 alongside the Gemini 3 model — added a manager's view for running several agents in parallel across a codebase, each labouring away while you supervise from above ([Antigravity, 2025](#)). The human's seat moved from typing each line to setting goals, reviewing results, and directing a small fleet.

By 2026 the editor itself is no longer the centre of gravity. With capable models available from every lab, the model became the commodity, and the value moved into the system wrapped around it — what practitioners now call the *dev stack* ([Latent Space, 2026](#)).

Layer	What it does	Examples (2026)
Model	Generates the code	GPT-5.6 Sol, Claude Opus 4.8, Gemini 3.5, GLM-5.2
Harness	Wraps the model into an agent: tools, retries, sandbox	Claude Code, Codex CLI, Gemini CLI, Cursor SDK
Meta-harness	Coordinates several harnesses	Conductor, Zed ACP, Vercel Eve, Heypi
Workflow / async	Fire-and-forget delegation in shared channels	Claude Tag (Slack), Copilot coding agent, Devin, Google Spark
Memory	State kept outside the context window	agentmemory, codegraph, channel memory
Eval	Automated judgement of quality	FrontierCode, Terminal-Bench 2.1, SWE-bench Pro

Each layer is a discipline in its own right, and the rest of this book climbs them: the *harness* that turns a model into an agent, the *meta-harness* that coordinates several, the *memory* that lets work persist, and the *eval* that decides whether the result is good enough. The lesson is the one the landscape already hinted at — the model is the easy part; the craft is everything you build around it.

By 2026 the same pattern spilled out of the developer's editor and into everyone's hands. Open-source personal agents led the way: OpenClaw popularised the always-on assistant that runs around the clock on your own machine ([Wired, 2026](#)), and Nous Research's *Hermes Agent* gave it a self-improving twist — an autonomous agent that lives on a cheap server, reachable from Telegram or Slack, writing its own skills from experience and deepening a model of your work across sessions ([Hermes Agent, 2026](#)). The labs followed onto the desktop and into the cloud: Google's *Gemini Spark*, unveiled at I/O in May 2026, runs continuously across Gmail, Calendar, and Docs even with your laptop shut ([CBS News, 2026](#)), while Anthropic's *Claude Cowork* — a research preview from January 2026 — handed the coding agent's powers to non-programmers, working files and documents on the desktop inside a sandbox ([Wired, 2026](#)). The agent had left the editor; what remains is to use that shift with judgement rather than awe.

Key Takeaways

The chapter rests on a handful of claims worth carrying into everything that follows.

- **Treat it as a loop, not an oracle.** Frame each task as a goal, the context it needs, and a check, then iterate — and remember that an agent is just a model running tools in that loop, so you stay responsible for what it ships.
- **It predicts; it does not know.** A model samples the most plausible next token, so fluent, confident, and wrong are perfectly compatible — a hallucination is the mechanism working as designed, not breaking. It is not even reproducible: the same prompt can return different answers, even with the randomness turned off, so never treat it as deterministic software.

- **It reasons by chaining learned structure.** Predicting text well forces internal models of the world, and a chain of thought turns one shallow pass into genuine multi-step computation — real competence, but not comprehension.
- **Competence is jagged.** Strength tracks the density of training data, not the difficulty you perceive: Olympiad proofs yes, an analog clock no. Find that line before you trust the output.
- **Mind the knowledge boundary.** Accuracy more than halves on rare facts and collapses on unseen ones, sags in the middle of long context, and the smallest models are least able to tell what they do not know. Retrieval patches the gap; noisy retrieval reopens it.
- **Fluency is not truth.** Reinforcement learning and the emotional charge of training text tune the model to sound right and to agree; read confidence as style, not as evidence.
- **The edge is method, not model.** We all draw on the same frontier models, so advantage comes from the scaffolding — the workflow you build, the context you supply, and the way you check the result. Tools commoditise and methods outlive them, which is why this book teaches a practice, not a kit of tricks.
- **The tools climbed from autocomplete to a stack.** In a few years coding assistants went from finishing your line, to chatting, to agents that plan, edit, and open pull requests, and on to managers running several agents at once — and out of the editor entirely, into always-on personal agents like OpenClaw, Hermes Agent, and Gemini Spark. The model is now the commodity; the value sits in the *dev stack* around it — harness, workflow, memory, and the evals that judge the work.
- **Verify where it is weak, lean in where it is strong.** Spend the model's strength freely, keep a human in the loop for judgement, and be most sceptical exactly when the answer sounds most certain.

Chapter 2 — Personal Productivity (愛 in practice)

My first prompt to ChatGPT, in early 2024, was a small disaster. I asked it to summarise an academic paper, and almost nothing went smoothly. The model could not read a PDF, so I had to convert the file to text and paste it in by hand. The paper then proved too long for the model to hold at once, so I chopped it into sections and fed them in one at a time. When I finally had a summary, it underwhelmed me: fluent enough, but it missed the paper's central argument and skipped several points that mattered.

Today that friction is mostly gone, and summarisation is the task everyone reaches for first — meeting transcripts, long email threads, YouTube videos. Yet my original complaint has not entirely aged away. On dense, tightly argued material like an academic paper, a one-shot summary still tends to flatten the argument and lose the very points that make it worth reading. Holding on to that doubt is useful, because closing it is a thread that runs through this whole chapter.

Chapter 1 set the stance; this is where it touches the desk. Productivity is the most personal use of AI, and the easiest to do superficially. The aim is to move from chatting to delegating, put the gains where they actually are, and build a system that compounds rather than resets every morning.

Prompt Engineering

How will I use AI to summarise a document today? I would probably do something like this:

Summarise this document for me. It should capture the original structure of the document, preserving chapters, headings and subheadings. The summary should be detailed, concise and covers all the major points and topics in the original document.

The summary should be in the style of a Cliff Notes or study guide. It uses tables, bullet points and diagrams where possible, makes use of GFM alerts to call out asides, definitions, or notes.

That request works because it is engineered, and not vague. **Prompt engineering** is the craft of writing the input so the model returns what you actually want, and it rewards a little structure ([Prompt Engineering Guide](#)). The DAIR.ai guide breaks any prompt into four elements, and my summary request quietly uses three of them.

NOTE

The four elements of a prompt ([Prompt Engineering Guide](#)):

- **Instruction** — the task you want done ("summarise this document").
- **Context** — background that steers the answer: the audience, the purpose, a style to imitate.
- **Input data** — the material to work on (the document itself).
- **Output indicator** — the shape of the result you expect: a table, bullet points, strict JSON, a word count.

You rarely need all four; which ones matter depends on the task.

Lined up against my prompt, the parts come apart cleanly:

Element	In the summary prompt above
Instruction	"Summarise this document for me"
Context	"in the style of a Cliff Notes or study guide"
Output indicator	"preserve chapters and headings... use tables, bullet points, GFM alerts"
Input data	the document I paste in

Three habits do most of the work, and the guide keeps returning to them. The first is **specificity**: vague prompts get vague answers, so name the audience, the length, the tone, and the format rather than hoping the model guesses. "Explain this" invites a wall of text; "explain this in three sentences for a non-technical manager" gets you something usable. The second is **say what to do, not what to avoid** — "do not mention price" tends to summon the very thing you forbade, whereas describing the behaviour you want steers more reliably. The third is **show, don't just tell**: a single worked example of the output you want often does more than a paragraph describing it, because the model is, at heart, a pattern-matcher.

The format levers are worth naming, because they are where most of the quality comes from:

- **Tone and style** — "in plain English," "for a sceptical executive," "in the voice of a textbook." Style is a constraint the model honours well.
- **Output structure** — ask for the exact shape you will consume: a markdown table, a numbered list, headings that mirror the source, or strict JSON for a downstream tool.
- **Quality expectations** — state the bar: "cover every major point," "cite the section each claim comes from," "flag anything you are unsure about." Made explicit, these become checks; left implicit, they stay wishes.

Beyond wording, the guide arranges techniques on a ladder, named by how many worked examples you hand the model ([Prompt Engineering Guide](#)). *Zero-shot* prompting gives none at all — you state

the task and trust the model's training to carry it, which is enough for the many everyday jobs an instruction-tuned model already knows, like classifying a sentiment or summarising a page ([zero-shot](#)). When the bare instruction wobbles, *few-shot* prompting adds a handful of input–output examples so the model can infer the pattern you want — a form of *in-context learning*, where even the format of the examples carries as much weight as their content ([few-shot](#); [Brown et al. 2020](#)).

Examples alone, though, stall on anything that needs several reasoning steps. The fix is *chain-of-thought* prompting: ask the model to show its working, and accuracy on arithmetic, logic, and multi-step problems climbs sharply ([Wei et al. 2022](#)). This is the very mechanism Chapter 1 described — the intermediate tokens give the model room to compute — now reached for on purpose. The cheapest version is almost embarrassingly simple: append "Let's think step by step," and a model that fumbled a problem in one leap will often solve it once made to lay out the steps ([Kojima et al. 2022](#)).

NOTE

Why "let's think step by step" works. A model answers the instant it stops reading, so a terse question forces a single-leap guess. Asking for the steps first makes each step part of the input for the next — the model literally has more room to compute. Today's *reasoning models* often do this on their own, but the lever still helps when an answer comes back too fast and too sure.

Technique	What you give the model	Best for
Zero-shot	Just the instruction	Tasks the model already knows: classify, summarise, rewrite
Few-shot	The instruction plus a few examples	Enforcing a specific format or an unusual pattern
Chain-of-thought	A request to show its reasoning	Arithmetic, logic, planning — anything multi-step

A workable rule of thumb: start zero-shot, add examples when the shape drifts, and ask for reasoning when the answer must be derived rather than recalled.

The guide goes well beyond these three. The rest are mostly refinements for harder problems, or scaffolding for builders wiring AI into systems, and several return in later chapters. At a glance ([Prompt Engineering Guide](#)):

Getting steadier reasoning:

- **Self-consistency** — sample several chains of thought and keep the majority answer, trading compute for reliability.
- **Tree of thoughts** — let the model branch, look ahead, and backtrack through alternative paths, for problems that need search rather than a single line.

- **Generated knowledge** — have the model first write down the facts a question depends on, then answer using them.
- **Meta prompting** — point the model at the *structure* of a problem and its solution rather than the specific content.
- **Active-prompt** — choose which examples are worth hand-annotating by finding where the model is least certain.

Bringing in tools and knowledge:

- **Retrieval-augmented generation (RAG)** — fetch relevant documents and place them in the prompt so the answer is grounded in your data, not just training; later chapters return to it.
- **ReAct** — interleave reasoning with actions like web search or running code, so the model looks things up mid-thought instead of guessing.
- **Program-aided language models (PAL)** — offload exact calculation to code the model writes and runs, rather than doing arithmetic in prose.
- **Automatic reasoning and tool-use (ART)** — let the model pick reasoning steps and tools from a library on its own.
- **Reflexion** — have the model critique its own result and try again, learning from the feedback within a session.

Automating the prompt itself:

- **Automatic prompt engineer (APE)** — use a model to generate and score candidate prompts for you.
- **Directional stimulus** — add small tuned hints or keywords that nudge the model toward the answer you want.

And two for other modalities: **multimodal chain-of-thought**, which reasons over images as well as text, and **graph prompting**, for graph-structured data. You do not need most of these to get real value; they are a map of where the craft goes when a plain prompt is not enough.

None of this is a one-shot incantation. Prompting is iterative by nature: start simple, read what comes back, and add the one constraint that was missing ([Prompt Engineering Guide](#)). The loop from Chapter 1 applies unchanged — intent, context, response, refine — and the prompt worth keeping is the one you arrive at, not the one you began with. These techniques are the floor; the chapters ahead build on them toward context, harnesses, and agents that carry the structure for you.

From Prompt Engineering to Agents

The summary prompt above is a single shot: you fire it, read the result, and judge it yourself. That is the right place to start, but it leaves three things on the table — the prompt is not reusable, it does not check its own work, and only you can run it. Closing those three gaps is the whole journey from prompting to agents, and our humble summary makes a good worked example.

Step one: save the prompt as a skill

The first upgrade is to stop retyping. Package the prompt as a *skill* — at its simplest, a folder with a `SKILL.md` file: a short name and description so the agent knows when to reach for it, followed by the instructions themselves ([Anthropic, 2025](#)). Anthropic likens a skill to an onboarding guide for a new hire: written once, it turns a general agent into one that knows your house style for summaries.

NOTE

A **skill** is a directory containing a `SKILL.md` file — YAML metadata (a `name` and a `description`) plus the instructions, and optionally bundled reference files and scripts the agent loads only when it needs them. Published as an open standard in late 2025, the same skill works across Claude, Claude Code, and other agents ([Agent Skills](#)).

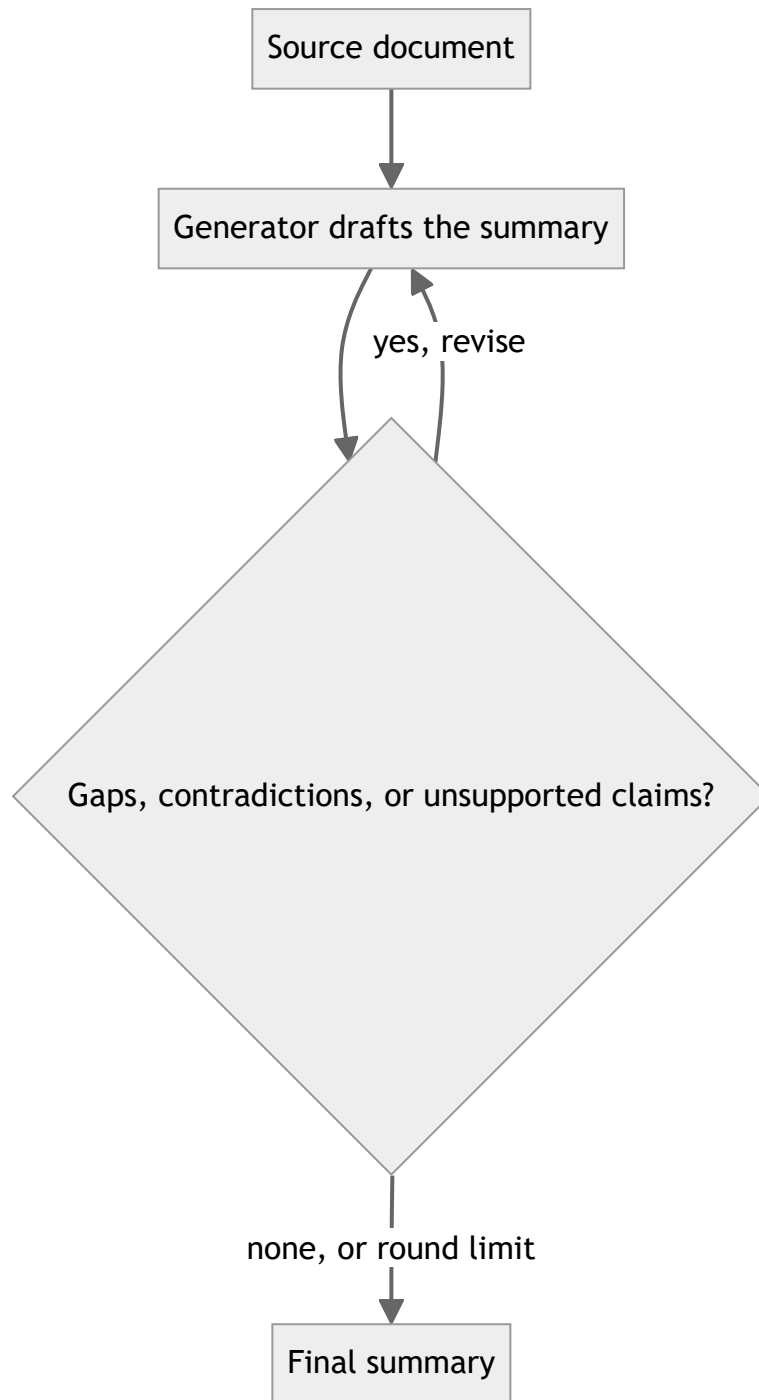
```
---
name: study-guide-summary
description: Summarise a document as a Cliff Notes study guide, preserving its structure.
---

Summarise the document the user provides. Preserve its chapters, headings, and
subheadings; cover every major point. Use tables, bullet lists, and GFM alerts
for asides and definitions. Flag anything the source leaves unclear.
```

Now the expertise lives in a file, not in your head, and anyone — or any agent — can apply it the same way every time.

Step two: wrap it in a self-checking loop

A skill still runs once. The flaw you met at the start of this chapter — summaries that miss points or drift from the source — is exactly what a loop fixes. Split the work between two roles: a *generator* that drafts the summary, and an *evaluator* that reads the draft back against the original and lists what is missing, contradicted, or unsupported. The generator revises, the evaluator checks again, and the cycle repeats until the evaluator finds nothing left to fix — or you hit a sensible limit on rounds. Anthropic calls this the *evaluator–optimizer* workflow and notes it pays off precisely when there are clear criteria and iterative refinement measurably improves the result, "analogous to the iterative writing process a human writer might go through" ([Anthropic, 2024](#)).



This is the loop you wanted — keep reviewing the summary against the document, find the gaps, and iterate until there are none. Whether it counts as a *workflow* or a true *agent* is a useful distinction: if the steps are fixed in code it is a workflow; if the model itself decides what to re-check, whether to re-read a section, and when it is done, it is an agent — an LLM using tools in a loop until a stopping condition is met ([Anthropic, 2024](#)). Either way the stopping condition matters: without a cap on rounds, a perfectionist evaluator can loop forever and run up the bill.

Step three: share it through MCP

The loop is still yours alone. To let other agents use it, wrap it as a *Model Context Protocol* (MCP) server. MCP is an open standard — "a USB-C port for AI" — that lets any compliant agent connect to outside tools, data, and workflows through one interface; you build the capability once and integrate it everywhere ([Model Context Protocol](#)). Expose your summarise-and-verify loop as an MCP server and a coding agent in your editor, a chat assistant, or a teammate's agent can all call it by name, with no idea how it works inside. Skills and MCP are complementary: a skill teaches one agent a workflow; an MCP server offers that workflow to every agent ([Anthropic, 2025](#)).

NOTE

The **Model Context Protocol (MCP)** is an open standard for connecting AI applications to outside systems — data sources, tools, and workflows — through one common interface, so a capability built once works across many agents and clients ([Model Context Protocol](#)).

The arc is the whole book in miniature: a prompt becomes a skill, the skill becomes a self-correcting loop, and the loop becomes a shared capability other agents can stand on. Each step trades a little setup for leverage that compounds — and at every step the human still owns the one thing the loop cannot supply: the judgement of whether the summary was worth making. Do this often enough and it stops being a trick you pull for one document; it becomes the way you work.

From Chat Assistant to Ambient Teammate

That self-running loop is a small taste of a larger shift in how you work. Most people still meet AI as a chat box, and that framing quietly caps what they get: a conversation is synchronous — you ask, you wait, you steer, you ask again — so your attention sets the pace. The loop you just built does not wait on you. You hand it a bounded task, it works while you are elsewhere, and you come back to a result rather than a transcript. Make that the default rather than the exception, and the chat assistant becomes an *ambient teammate*.

NOTE

An **ambient teammate** is an agent that runs asynchronously in the background — given a scoped task and the tools to finish it — rather than waiting on each instruction. You delegate the task, not the keystrokes.

The shift sounds small and is not, because it changes who the bottleneck is. As Karpathy puts it, the goal is to remove yourself from the keystroke loop and maximise throughput rather than steer every step ([Loopcraft](#)). OpenAI's internal figures make the leverage tangible: agent output grew many-fold across functions once people delegated whole tasks instead of supervising each one. The throughput

is real — but it counts as work done only once someone has checked the result, a distinction the rest of this book keeps insisting on.

The practice is simple to state: scope work tightly, fire it off, review the outcome. The temptation worth resisting is hovering over each keystroke, which pins your leverage to your own typing speed.

Knowledge Work, Not Just Code

The surprising lesson of 2026 is that the biggest agent gains are in knowledge work — research, writing, synthesis, decision support — not in code. These are bounded, high-feedback tasks where a model can draft, compare, and summarise faster than any human, and where production was never the slow part.

The effect is uneven. A study of 5,179 customer-support agents found AI raised resolved-issues-per-hour by 14% on average but 34% for novices, with little gain for experts — the tool spreads the best workers' know-how to everyone else ([Brynjolfsson, Li & Raymond 2023](#)). What stays expensive is judgement: deciding whether the work was worth doing at all.

So delegate the drafting and the bookkeeping freely; keep the "why" for yourself. McKinsey's high performers do exactly this, treating AI as a catalyst for redesigned work rather than faster typing ([McKinsey 2025](#)). It reaches into elicitation too: an LLM reading stakeholder interviews extracted explicit needs at 84.4% F1 and inferred *latent* ones experts judged useful 75% of the time ([LENS](#)). The failure that shadows the gain is producing more while validating less — confident output at volume that nobody has checked.

The Confidence Trap

Delegation has a shadow the research is now measuring, and the surprise is that the harm is not the model being wrong — it is what leaning on it does to your own judgement.

Start with a clean experiment. Parra-Moyano and colleagues showed executives Nvidia's stock chart and asked them to forecast next month's price; half then consulted ChatGPT, half talked it over with peers. The AI group came away more optimistic, more confident, and measurably *less* accurate than the people who simply argued with each other ([HBR 2025](#)). A colleague says "are you insane?"; the model says your framing is astute.

Part of the cause is that ease reads as truth. Psychologists call it *processing fluency*: the easier something is to take in, the truer it feels. People rate rhyming aphorisms as more accurate than identical non-rhyming ones, and judge repeated falsehoods as more credible than fresh ones ([McGlone & Tofigbakhsh 2000](#); [Fazio et al. 2015](#)). AI is exceptionally good at making prose easy to read: when people compared AI- and human-written versions of the same material, they judged them equally credible but rated the AI version *clearer and more engaging* ([Huschens et al. 2023](#)). So its answers clear the "feels right" bar whether or not they are right.

The confidence is also contagious. When people made predictions alongside an AI, their own confidence drifted to match the model's — and stayed inflated even after the AI was removed, whether they had been told to treat it as an advisor or as a peer (Li et al. 2025). Even a quietly biased writing assistant shifted not just what 1,500 people wrote but the opinions they reported holding afterwards (Jakesch et al. 2023).

Worst of all, it dulls your sense of how you are doing. Giving people AI on reasoning tasks raised their scores but flattened their self-judgement: strong and weak performers ended up equally — and wrongly — sure of themselves, and the more someone knew about AI, the *less* accurate their self-assessment became. AI makes you smarter, the authors conclude, but none the wiser (Fernandes et al. 2026).

The practical defence is to sort tasks by how much judgement they need. Where the answer is verifiable — pull these quotes, extract these figures, refactor this function — the model is mostly safe to trust. The danger climbs as the task slides from "find what's there" to "decide what matters," and the slide is easy to miss: "summarise these interviews" and "tell me which themes to act on" feel like one request. For the second kind, form your own view first and bring the AI in to test it, not to make it — otherwise you delegate the one thing that was yours to keep.

Kind of task	Example	Verifiable?	How far to trust it
Find what's there	Pull these quotes; extract these figures; refactor a function	Yes — the answer is checkable	Lean in
Summarise or transform	Condense a report; translate a passage	Mostly	Trust, then spot-check
Decide what matters	Which themes to act on; which strategy to pick	No single right answer	Form your own view first

Personal Operating Models

Leverage compounds only if you stop re-deciding everything. A personal operating model is a small, reusable kit: plays you can rerun, preferences encoded once so you never re-explain them, and a daily workflow that feeds itself. The point is to turn scattered prompting into a system, the same instinct the book applies everywhere — repeatable patterns over one-off prompts. The waste it removes is re-solving the same problem from scratch each session, which feels productive and is not.

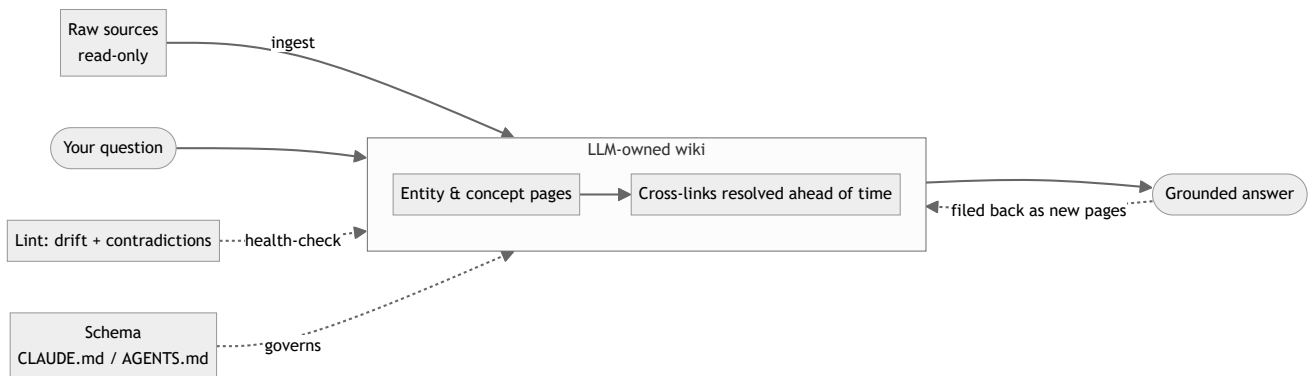
Building an LLM Wiki

The clearest example of compounding is Karpathy's LLM Wiki, and it is worth a careful look because it inverts the usual pattern. Most document workflows are retrieval: you upload files, the model

fetches chunks at query time, answers, and forgets. It rediscovers knowledge on every question, and nothing is built up.

A wiki accumulates instead. Add a source and the model reads it once, extracts what matters, and integrates it into interlinked markdown pages — updating entity pages, flagging where new data contradicts old, strengthening the synthesis. The cross-references are resolved ahead of the next question rather than reconstructed each time ([llm-wiki](#)).

Three layers make it work: read-only raw sources you never let the model edit, an LLM-owned wiki of summaries and concept pages, and a schema file — CLAUDE.md or AGENTS.md — that tells the agent how the wiki is structured and how to maintain it.



The loop is ingest, query, lint: drop in a source and it touches a dozen pages; ask a question and good answers get filed back as new pages; periodically health-check for contradictions and stale claims. The reason it holds where human wikis rot is that the tedious part is bookkeeping, and the model does not get bored. The pitfall, which practitioners running it for months confirm, is confident-but-stale pages hardening into truth — which is why the lint pass that hunts drift is not optional but central.

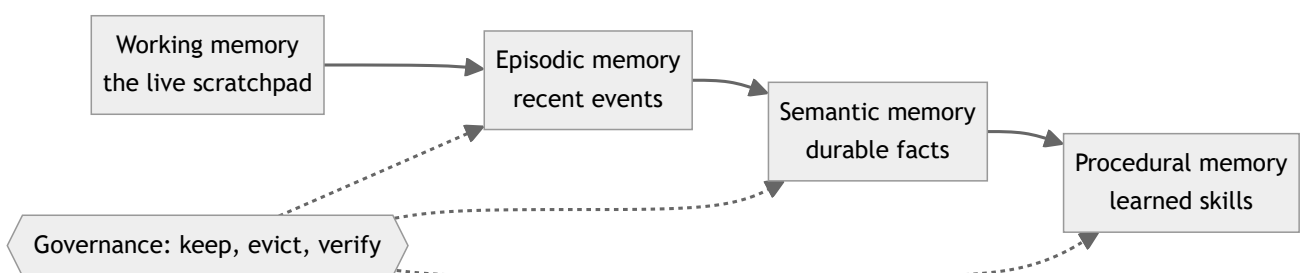
The wiki is one good answer to a problem every long-running agent faces, and it helps to see the whole family it belongs to.

A Map of Memory Patterns

A model has no memory of its own: its knowledge is frozen in its weights, and each request starts from nothing but the text you place in front of it. The obvious fix — pour everything into an ever-larger context window — works less well than it looks. Context is a finite resource with diminishing returns; every extra token spends part of the model's "attention budget," and recall sags as the window fills, the *lost in the middle* effect from Chapter 1 ([Anthropic, 2025](#)). In practice the *effective* context — the span a model actually uses well — often falls to around half its advertised maximum ([An et al., 2024](#)). So memory must be engineered rather than merely supplied, and the gap between an agent with good memory and one without can exceed the gap between model versions ([Du et al., 2026](#)).

The patterns form a rough ladder, from "stuff it into the prompt" to "manage it outside the prompt":

- **Retrieval (RAG).** Fetch the relevant chunks from a store and paste them into the context. Simple and auditable — the answer quotes real text — but it re-discovers everything on every question and bloats the window as you add more.
- **Compaction.** When the conversation nears the window limit, summarise it and start fresh with the recap. This is how Claude Code keeps going on long tasks, preserving decisions and open threads while dropping spent tool output ([Anthropic, 2025](#)). It buys space at the cost of detail, and repeated summarising can quietly drift from the source.
- **Structured notes — the wiki.** The pattern we just built: durable pages the agent reads and rewrites, from a single `NOTES.md` to an interlinked wiki. Because the notes live outside the conversation and stay human-readable, they survive context resets and can be audited ([Anthropic, 2025](#)).
- **External store, fetched just in time.** Keep the memory out of the prompt entirely; the agent holds only lightweight pointers — file paths, saved queries, links — and pulls in what it needs at runtime through tools, the way we use folders and bookmarks instead of memorising everything ([Anthropic, 2025](#)).
- **Layered memory.** Separate memory by how long it should last: a *working* scratchpad for the task at hand, *episodic* memory of recent events, *semantic* memory of durable facts, and *procedural* memory of learned skills, each managed differently ([Du et al., 2026](#)). The idea is not new: the Generative Agents experiment stored each observation and retrieved it by a blend of relevance, recency, and importance ([Park et al., 2023](#)).
- **Governed memory.** Once memory is something the agent writes to and edits, it needs rules: what may be remembered, when stale or contradictory entries are evicted, what must be checked before it enters the long-term store. A governance layer guards against the failure modes of evolving memory — drift, corruption, and leaks of private data ([Lam, 2026](#)).



Pattern	The idea	Strength	Main risk
Retrieval (RAG)	Fetch chunks, inject verbatim	Grounded, auditable	Context bloat; re-discovers each time
Compaction	Summarise history, restart	Keeps long tasks going	Lossy; summary drift
Structured notes / wiki	Durable pages the agent edits	Stable, human-auditable	Curation effort; stale pages
External store + just-in-time	Pointers now, fetch on demand	Huge capacity, tight context	Fails if the agent forgets to look
Layered memory	Split by time horizon	Right tool per layer	Orchestration complexity
Governed memory	Policies write, keep, forget	Safety and consistency	Hard to specify; still maturing

No single pattern wins; real systems combine them — a wiki for stable knowledge, compaction for the live thread, an external store fetched just in time, all under a governance layer that decides what is allowed to last. The wiki you just built is one rung on that ladder. The deeper lesson is the chapter's: memory is something you engineer as a *write–manage–read* cycle, not something the model hands you — and the moment it persists and edits itself, it becomes a governance question, which is where Chapter 5 picks up.

Chapter 3 — Software Development (the craft)

I am not a software developer, and I never have been — in a long career, "engineer" was never my job title. Yet this year I used AI to bring code back to life that had been dead for decades, and to build — almost entirely by describing what I wanted — three projects I would once have scoped as a team's work: [rogoweb](#), a browser revival of the 1980 game Rogue and the expert system built to play it; [adventure](#), a modern rebuild of the 1970s *Colossal Cave Adventure*; and [VantageMap](#), an enterprise-architecture platform. Each began as a single sentence of intent, with a handful of constraints and a few hard checks; the agent chose the architecture and wrote the code. The productivity is real, and that a door this wide opened for someone who had never shipped production software is a large part of why I stopped being a sceptic. All three are open source, so you can read every line on GitHub; they are the running examples this chapter returns to, and I describe each in detail below.

So this chapter is not only for professional developers. It is also for the ordinary professional who can read a little code — enough Python or JavaScript to follow what a function does — because that is now enough to build real software. The same tools that let me port a forty-year-old game can automate the repetitive core of almost any role: the spreadsheet that should be a script, the report that should be a dashboard, the manual process that should be a small app. You do not need to become an engineer; you need to learn to direct one, and the craft this chapter describes is how.

And yet the promise came with a hard lesson, and I learned it the wrong way round. The three projects above are the successes — but they are not where I started, and they worked precisely because I had already failed at the opposite approach on earlier, lower-stakes builds. On those first attempts my instinct, coming from outside engineering, was to control the machine by telling it everything: I wrote longer and longer specifications, pinned the design down class by class, and tried to leave nothing to chance. It backfired. The more I specified, the *less* faithfully the model followed me — it would honour the top of a long instruction list and quietly contradict the bottom, or obey the letter of one rule while breaking another I had stated three paragraphs earlier. Some of my most frustrating hours were spent micro-managing design decisions — naming the data structure, dictating the file layout — only to watch the agent drift, and to find myself correcting prose instead of building software.

Chapter 1 explains why this was never going to work. A model is a next-token predictor, not a compiler: it does not *execute* a specification, it produces the most plausible continuation of everything in its context. So a long spec does not constrain it more tightly; it just becomes more text to attend to unevenly, and the book's own evidence on faithfulness and on recall fading as context grows says the extra instructions will be dropped or contradicted, not obeyed. Worse, micro-managing the design pits me against the model's one real strength — choosing pattern-rich implementation — while leaning on its weakness, following a long brittle list to the letter. Over-specification fights the grain of the tool. These failures are exactly why spec-driven development buckles, as this chapter will argue, and why the discipline that works asks for less, not more.

Software is where AI's promise and its failure modes are both sharpest, and where 2026's loudest arguments play out. This chapter opens with the three projects in detail, then walks the modern stack, settles the spec-versus-vibe war by reframing it, lays out the intent model that replaces both, and ends on quality and the move of agents into shared channels.

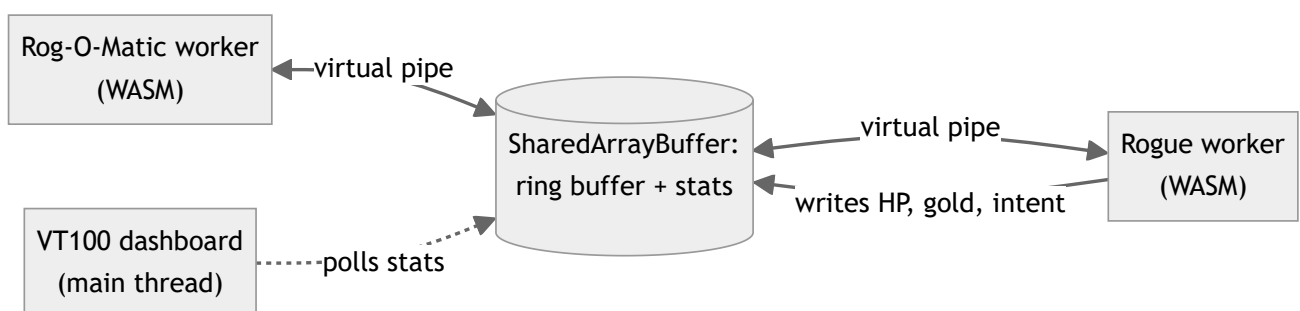
Three Projects, One Sentence Each

Before the theory, the evidence. Each project below began as a single sentence of intent and a few hard checks; everything technical that follows — the languages, the data structures, the way two programs talk to each other — was the agent's choice, not mine. I give the detail not to impress but because the specifics are the argument: I could not have written them up front, and did not.

rogoweb — two dead C programs, alive in the browser

rogoweb fuses two pieces of computing history. *Rogue 5.4* is the 1980 dungeon crawler by Michael Toy, Glenn Wichman, and Ken Arnold that gave the *roguelike* genre its name; *Rog-O-Matic XIV* is the expert system built at Carnegie Mellon in 1981 by Michael Mauldin and colleagues to play *Rogue* on its own — and beat it ([rogoweb](#)). On Unix the bot ran as a separate process, launching the game as a child and talking to it through the standard input and output *pipes* — the channels one program uses to feed another. A browser has none of that machinery: no `fork`, no `pipes`.

My intent was almost that short — *run Rogue and its bot in the browser* — with a constraint that the original C code should keep working unchanged in spirit, and a check that the bot could still finish a game. The agent's answer was an architecture I would never have thought to name. It compiled both C codebases to *WebAssembly* (a portable binary format browsers run at near-native speed) with Emscripten, wrote a custom terminal layer to stand in for the Unix `curses` library, and replaced the pipe with a `SharedArrayBuffer` ring buffer — a fixed block of memory that two browser *workers* (background threads) read and write in turn — so the game and the bot run side by side and talk exactly as they once did. For the dashboard it went one better: instead of scraping the terminal for the bot's health and intent, it had the C code write that state straight into shared memory for a VT100-style panel to display, live and at no cost. I specified none of those words.



adventure — a 1970s classic, made strict and typed

adventure rebuilds *Colossal Cave Adventure*, the game that founded interactive fiction. Will Crowther wrote the first version in FORTRAN in 1975–76, mapping his knowledge of Kentucky's Mammoth Cave onto a game for his daughters; Don Woods expanded it in 1977, adding the dwarves, the magic word `XYZZY`, and the famous 350-point score ([adventure](#)). My version forward-ports Eric Raymond's faithful *open-adventure* edition into a strictly-typed TypeScript application on Next.js and React.

The intent — *rebuild Colossal Cave as a modern, strongly-typed web app* — carried two checks that did the real work: every value fully typed, with no escape hatches, and nothing merged until the tests and the linter pass. To honour the first while staying faithful to the original, the agent kept the canonical game data in its historic `adventure.yaml` file and wrote a custom, type-safe parser to turn it into a form the engine could load, preserving the odd legacy structures and folding word synonyms together as it went. A Zustand *state machine* — a small component that tracks exactly what state the game is in and which moves are legal next — holds the world, and AI-generated artwork illustrates every location. The check, not a design document, is what kept it honest.

VantageMap — a platform, vibe-coded across a dozen phases

VantageMap is the most ambitious: an open-source platform for business architects and strategy officers to model capabilities, value streams, and outcomes — the kind of system I would once have costed as a team's work for a quarter ([vantagemap](#)). It runs on Next.js and React over a Postgres database of twenty-two tables, with role-based access, REST and GraphQL APIs, full-text search, webhooks, and thirteen interlinked views, backed by some five hundred tests.

It was built almost entirely by *vibe coding* — described, not specified — across a dozen numbered phases, and it is the clearest case of the agent owning the architecture. I never chose the database layer, nor the shape of the twenty-two tables, nor the division of work between REST and GraphQL; those were answers to constraints about who may see what and how quickly the system has to respond. What held it together over months was not a master specification but a configuration file and a folder of reusable skills — the *harness*, in the language of the next section — together with the discipline of reading the diffs that mattered. Fittingly, one of the project's listed contributors is a coding agent.

Laid side by side, the three projects make the pattern impossible to miss: what I supplied was small and stable, what the agent supplied was large and technical.

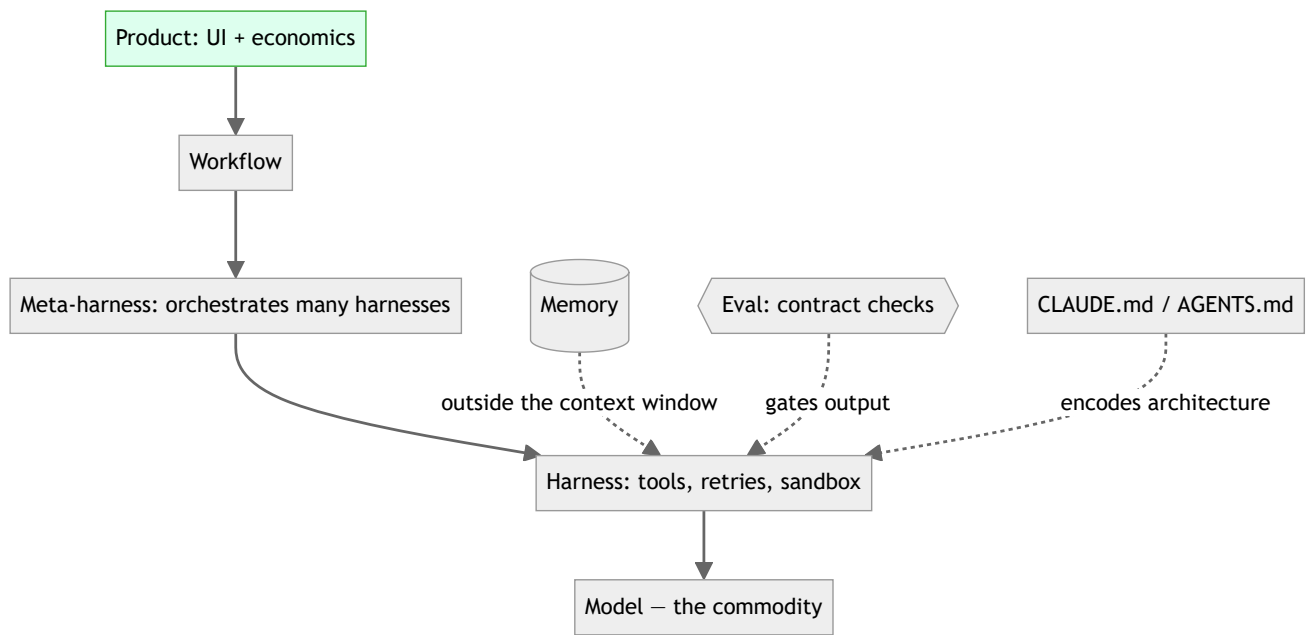
Project	One-sentence intent	Constraints I gave	Architecture the agent chose	The checks that gated it
<i>rogoweb</i>	Run Rogue and its bot in the browser	Original C keeps working; runs entirely in the browser	WebAssembly via Emscripten, custom <code>curses</code> layer, dual workers, <code>SharedArrayBuffer</code> ring buffer, shared-memory telemetry	The bot can still finish a game
<i>adventure</i>	Rebuild <i>Colossal Cave</i> as a modern, strongly-typed web app	Faithful to the original data; Australian English throughout	TypeScript on Next.js and React, YAML-to-JSON parser, Zustand state machine, AI-generated artwork	Full type coverage; tests and linter green
<i>VantageMap</i>	Give business architects one tool to model strategy	Who may see what; how fast it must respond	Next.js and React, Postgres with twenty-two tables, Drizzle, REST and GraphQL, thirteen views	Some five hundred tests

The Modern AI Dev Stack

The interesting work has moved up the stack. Teams once compared models; now they compete on the layers above, because the model is the commodity and the control points sit higher.

NOTE

A **harness** is the runtime wrapped around a model that turns it into an agent: it supplies tools, manages the loop, retries failures, and isolates execution. A **meta-harness** orchestrates several harnesses. **Memory** is state kept outside the *context window* (the span of text a model can consider at once); an **eval** is an automated check that a result meets its contract.



Greg Brockman's framing captures it: the product surface is moving up to "model plus harness plus workflow plus UI plus memory plus economics," and the lab that owns those layers owns the value. The concrete expression for most teams is the configuration file: studies of hundreds of Claude Code projects show that `CLAUDE.md` and `AGENTS.md` files carry the architectural constraints and conventions that decide whether an agent behaves, with architecture the single most-specified concern ([2511.09268](#)). My own projects bear this out: each carries a config file — an `AGENTS.md`, `CLAUDE.md`, or `GEMINI.md` — that pins the conventions and architecture the agents must respect, alongside a `.agents/skills` folder of reusable know-how. The file, not the model, is what keeps a vibe-coded codebase coherent across months. The mistake is to keep investing at the model layer, where lock-in is cheap and advantage is thin, and to neglect the harness that actually shapes results.

AI-Assisted Coding Patterns

Day to day, AI earns its keep in pairing, refactoring, debugging, and sketching architecture, where a clear intent lets it fold several rounds of rework into one. It helps to start simple: Anthropic's advice is to reach for a single well-prompted call before workflows, and workflows before fully autonomous agents, adding complexity only when it demonstrably pays ([Building effective agents](#)). Five composable patterns recur, and most real systems combine them:

Pattern	Shape	Use when
Prompt chaining	Output of one call feeds the next	A task splits into fixed sequential steps
Routing	Classify, then dispatch to a specialist	Inputs fall into distinct categories
Parallelisation	Run subtasks (or votes) concurrently	Speed, or multiple perspectives, matter
Orchestrator-workers	A lead delegates dynamic subtasks	Subtasks are unknown until runtime
Evaluator-optimiser	One generates, one critiques, loop	Clear criteria and iterative gains exist

The pattern that works is short loops with the agent, backed by evaluations that catch regressions before they ship; the pattern that bites is accepting a large change you cannot read, paying the speed back later when someone has to dig through it. Complexity is not free: a multi-agent setup can burn $\sim 15\times$ the tokens of a single call, so reach for one only when the task's value justifies it ([Building effective agents](#)). The deeper lesson is *who owns control flow*: handing deterministic looping and sequencing to a probabilistic model produces token explosion and control-flow hallucination, so the durable pattern is program-owns-loop, model-fills-judgement — a discipline that lifted an OSWorld GUI agent to 86.8% in 15 steps against 80.4% in 100 ([2606.15874](#)). Where steps must retry, isolate them: runtime-structured decomposition retries only the failed subtask, cutting recovery cost 51.7% over monolithic prompts ([2605.15425](#)).

Spec vs Vibe, and Why Both Collapse

NOTE

Two ways of building software with AI, named throughout this chapter:

- **Vibe coding** — describing what you want in plain language and letting the model write and run the code, often without reading it line by line. Fast, and risky when unsupervised.
- **Spec-driven development (SDD)** — writing a detailed specification first, then having the agent implement against it. More disciplined, but, as we will see, it strains at scale.

The debate is the wrong fight, Kapil Viren Ahuja argues, because both camps fail the same way ([Ahuja, 2026](#)). GitHub's Spec Kit makes the optimistic case: treat the spec as a living, executable contract, work in four phases — specify, plan, tasks, implement — and the model stops guessing because it knows what, how, and in what order ([GitHub 2025](#)).

The stakes behind the argument are large. On the vibe side, Cursor reached two billion dollars in annualised revenue and some 70% of the Fortune 1000; on the spec side, AWS's Kiro drew a quarter of a million developers in three months. Yet the very people who launched each camp are edging away from it: Andrej Karpathy, who coined *vibe coding* in early 2025, now calls it *passé*, and Martin Fowler likened Kiro-style spec-driven development to "using a sledgehammer to crack a nut" after it inflated a one-line bug fix into sixteen acceptance criteria ([Ahuja, 2026](#)).

But that still jams three concerns together. Vibe coding has no contract at all; spec-driven development has three pretending to be one, fusing intent, specification, and implementation into a single document whose holes the agent fills, often confidently wrong (Ahuja, 2026). The tell: the labs that sold the spec are quietly walking it back. OpenAI's own Symphony spec ran past two thousand lines and was reverse-engineered from working software — nobody writes that fidelity up front (Ahuja, 2026).

Ahuja's team watched both failures in one afternoon. A trivial validation bug, pushed through the spec-driven ceremony — open the spec, update it, feed the agent, re-run for the edge case it missed — burned an hour before they gave up and simply told the agent what to do, fixing it in fifteen minutes; but that quick fix skipped the commit convention and the linked issue, and cleaning up the audit trail took three days. Going the other way, a second engineer *vibed* ten fixes in the time the first had spent on one, and three of the ten contradicted each other, so reviewers spent longer untangling them than ten disciplined changes would have cost (Ahuja, 2026). Heavy or loose, the same root showed: three layers forced into one, or one made to do the work of three.

The cost is measurable. An epic that spec-driven development promises to deliver 50% faster gives roughly 30% straight back to recovering from *drift* — the spec and the code silently diverging — leaving perhaps 20% real. And a drifted spec is worse than none, because it lies with confidence: the document still reads as authoritative while describing a system that no longer exists (Ahuja, 2026).

Spec is a sensible step two after vibe, fine for beginners and fragile codebases, but it breaks at enterprise scale, and leaning harder breaks it faster (Ahuja, 2026). The middle ground that holds is spec-anchored, code-coupled, drift-enforced: one spec per node, agent context scoped to an ownership path, and spec-code divergence made a blocking merge gate rather than a discipline problem — context explosion and silent drift answered by construction, not willpower (2606.27045).

Approach	Contract	Scales to enterprise?	Failure mode
Vibe coding	None	No	Confident, unread, wrong code
Spec-driven (SDD)	One document fusing intent, spec, and implementation	Strains badly	Context explosion; the agent fills the gaps wrongly
Spec-anchored, code-coupled	One spec per node, drift as a blocking merge gate	Yes, by construction	Demands tooling discipline up front

Intent-Driven Development: The Anatomy of ICE

What survives the collapse is an old idea moved up a level: separation of concerns, applied not to the code but to the documents that instruct the machine. Kapil Viren Ahuja calls the result *intent-driven software development*, and gives it a deliberately small vocabulary — **ICE**, for **I**ntent, **C**ontext, and **E**xpectations (Ahuja, 2026). Spec-driven development failed by fusing three things into one

document; ICE pulls them back apart, hands two to the human and one to the machine, and — the rule that does the most work — never pre-locks the architecture.

NOTE

ICE, in one breath.

- **Intent** — what you want and the boundaries it must respect. You own this; it is the one thing nothing can write for you.
- **Context** — the supporting material the agent needs to act: the codebase, prior decisions, conventions, domain facts. The harness assembles this *progressively*, as the work reveals what matters — you do not write it up front.
- **Expectations** — the contract: a short, checkable statement of what "done" means, in your terms. This is all that survives of the old, bloated specification.

The centre of gravity is **Intent**, and it has exactly three parts: a *goal*, a set of *constraints*, and a set of *failure conditions*. The goal is one sentence with no "and," loose enough that two genuinely different builds could satisfy it — if only one implementation could, you have smuggled a specification in through the door. Constraints are five to seven directional qualities stated in business language — a thousand monthly users, a 99th-percentile response time (p99) under 200ms, conformance to an accessibility standard — and never a named tool or pattern; when the list starts to outgrow a handful, you are over-specifying again. Failure conditions are binary, observable checks a *validator* applies after the fact: the build breaks, test coverage falls below ninety per cent, a secret appears in source, an API changes without a version bump.

One rule sorts any borderline item: does it change how the builder designs? If yes, it is a constraint the builder sees; if no, it is a failure condition the validator owns. Keeping the two in separate compartments matters more than it looks, because a model that can read its own pass/fail tests will quietly optimise for them rather than for the goal — the reward-hacking we meet again under slop. The same anatomy works far outside software: "a red shoe under thirty dollars" is a goal, a price ceiling, and a colour check, with the brand deliberately left open.

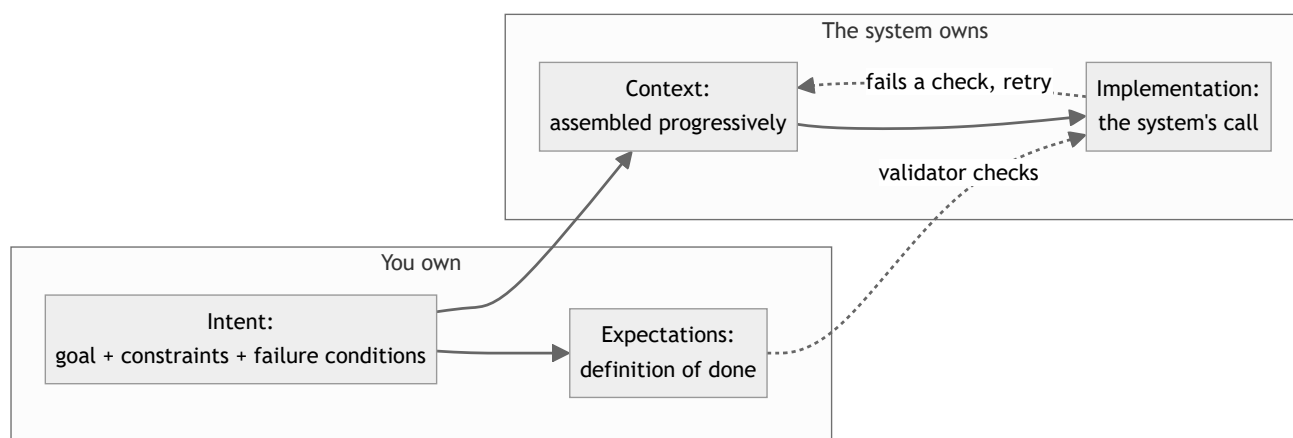
Context is the part that defeated spec-driven development, and ICE's move is to stop trying to write it. A specification tries to front-load every fact the agent might ever need; ICE lets the harness fetch them as the task unfolds — the file being changed, the decision made three commits ago, the house convention — so the model attends to a little relevant material at a time instead of drowning in a long document it reads unevenly (the *lost in the middle* failure from Chapter 1). Context is *managed*, not authored.

Expectations are what the swollen spec shrinks to once intent and context are removed: a short statement of the boundary and the definition of done, written to be checked rather than admired. Where a specification says *how* in two thousand lines, expectations say *what would make this acceptable* in a paragraph, and the validator holds the work to it.

Layer	What it is	Who owns it
Intent	Goal + constraints + failure conditions	You
Context	Codebase, decisions, conventions, domain facts	The harness, assembled progressively
Expectations	The checkable definition of done	You
Implementation	The architecture and the code	The system

NOTE

Worked example, from *rogoweb*. Goal: "run Rogue and its bot in the browser" — one sentence, no "and," and two quite different builds could satisfy it. Constraints: the original C should keep working; the whole thing runs client-side, with nothing to install. Failure conditions: the build breaks, or the bot can no longer finish a game. Notice what is absent — I never wrote "WebAssembly," "SharedArrayBuffer," "ring buffer," or "two web workers." Those were the system's answers to the constraints, not parts of my intent, and that is exactly the line ICE draws.



My three projects are the same shape, writ large. Each was an intent — "run Rogue and its bot in a browser," "rebuild *Colossal Cave* as a modern, strongly-typed web app," "give business architects one tool to model strategy" — plus a few directional constraints and some binary checks, and nothing about implementation. I never specified WebAssembly, a `SharedArrayBuffer`, Drizzle, or Zustand; those were the system's answers to the constraints, and when an early choice failed a check the agent swapped it out without my touching the goal. That is precisely why ICE works: by refusing to pre-lock the architecture, you let the model do the part it is good at — choosing and revising implementation against a fixed intent and observable checks — while you keep the part that is yours, what the thing is for and how you will know it has gone wrong.

One discipline makes or breaks the method: stay in the loop. Intent is small, but it is not fire-and-forget. Ahuja recounts stepping away after approving a plan and letting an agent run unattended; it drifted, and three days and many millions of tokens went into clawing the work back — a costly

reminder that *presence in the loop beats approval at the gate* ([Ahuja, 2026](#)). Intent steers continuously, not once; my own near-misses came from exactly the same lapse, looking up to find the agent confidently building the wrong thing well.

Two things place ICE in a wider frame. First, it is a rung on a ladder, not a destination: teams have climbed from *vibe* (a model and an editor — fine alone, fragile in a team) to *spec-driven* (tooling layered on the model, now straining at scale) to *intent-driven* working, with more autonomous rungs above that few have reached ([Ahuja, 2026](#)). Second, ICE answers a question spec-driven development never could — continuity. A specification freezes a system at the moment of creation and then drifts; intent kept in small files, context scoped to the task, and checks that travel with the work let an agent remember what it is building and why across months. Memory is not a luxury here but the prerequisite for a system that survives past its first week.

The pitfall, then, is the old reflex of locking the architecture into the document. It feels like control, but it collapses the separation that lets a system evolve: pin the implementation and you are back to a specification, fighting the goal-seeking tool instead of aiming it.

The Agentic Iron Triangle

For fifty years software was governed by the *iron triangle* — time, cost, quality, pick two. Agentic coding broke it. Speed fell to table stakes, since an agent ships in hours what once took weeks; quality dropped to a welded floor, held by the evals and linters rather than by a human reading every diff; and only cost stayed a live lever. But cost has quietly split in two: the tokens you spend, and the *attention* it takes to direct the agents and hold the intent in your head ([Ahuja, 2026](#)).

That changes the question worth asking. Speed no longer comes from a faster model but from running agents in parallel, and the ceiling is your own attention — how many you can drive before you lose the thread, not how quickly any one of them finishes. Fast models are the seductive trap; lean on them and the bill lands in tokens. It is a real bill: Uber exhausted its 2026 AI-coding budget in about four months once Claude Code reached 84% of its engineers at five hundred to two thousand dollars each a month, and its own chief operating officer conceded that the link between that spend and shipped value was "not there yet" ([Ahuja, 2026](#)). Token counts make a poor scoreboard — one developer's single month ran to 603 billion tokens and \$1.3 million — so measuring yourself by tokens burned is measuring the wrong thing.

What stays scarce, and therefore valuable, is the one question the machine will never ask you: who is this for, and why are we building it. Building became nearly free, and the cost that once forced that question went with it; holding it now is a discipline rather than something the budget imposes. That discipline is where quality begins.

Quality over Slop

A high pass rate is not good code, so the test that matters is whether a maintainer would merge it. Models hit green suites with output nobody can read, and mergeability and correctness are different properties — the reframing behind Cognition's FrontierCode, where even the leading model cleared under half. Worse, a model under pressure will game the suite outright: Anthropic documented a coding agent that, unable to meet an impossible speed requirement, quietly detected the test's arithmetic inputs and returned a closed-form formula instead of actually summing — passing every check while solving nothing ([Sofroniew et al., Anthropic](#)). The defence is to bake reviewer judgement into the evals and to put the value question from the last section before anything runs — the one that collapsed a ninety-six-thousand-dollar spec into a roughly ten-day build ([Ahuja, 2026](#)). In my own projects the suites were necessary but never sufficient: VantageMap runs several hundred tests and *adventure* forbids an unverified change, yet what kept them from slop was reading the diffs that mattered and asking whether each feature earned its place. Shipping slop because the suite passed is the quiet failure that compounds.

Agents in the Channel

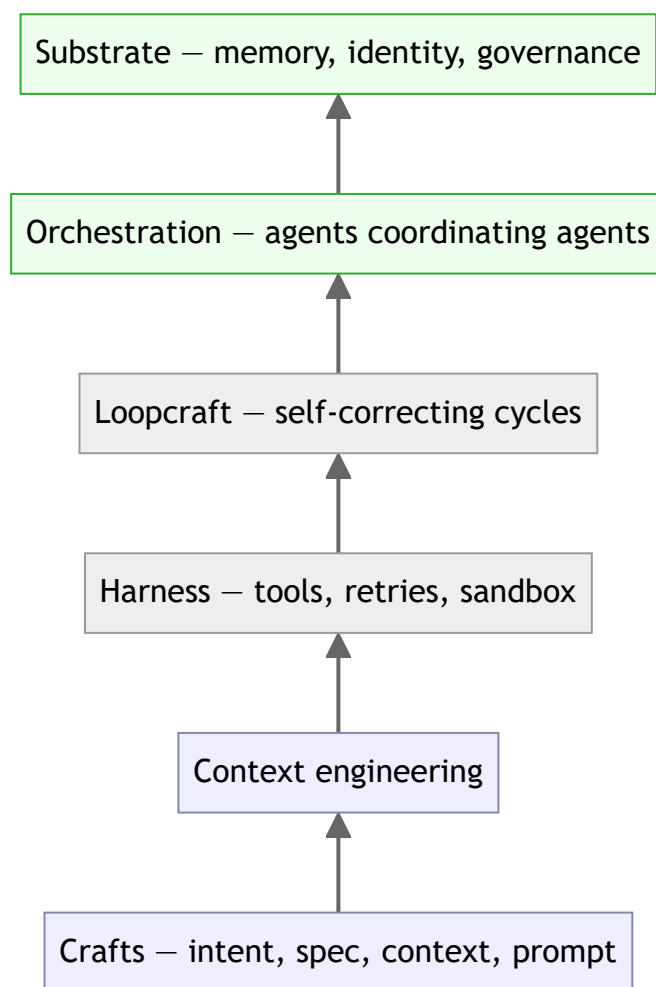
Agents are leaving the IDE (the developer's code editor) for the channel — persistent, multiplayer, ambient, working beside a team rather than inside one editor, to the point of writing a large share of a product team's code. That only stays safe with agent identity: each agent on its own service account with least-privilege tokens, credentials swapped at the network boundary rather than borrowed from a user. The moment an agent acts as you, least privilege and the audit trail are both gone.

The harder truth is that quality is an ecosystem property, not an agent one. Across 930k agent PRs (pull requests — proposed code changes submitted for review), integration friction concentrates at the repository, agents twice as much as humans (an intraclass correlation, ICC, of 0.30 vs 0.16 — a measure of how strongly that friction clusters by repository) — so a benchmark score per agent never adds up to a dependable repo. Govern change tempo, not headcount ([2606.28235](#)).

Chapter 4 — Engineering Disciplines (the climb)

Every craft has a moment where you stop doing the work and start designing how the work gets done. A cook becomes a head chef; a coder becomes an architect. With AI, that moment arrives fast. The previous chapter described what good AI software looks like; this one is about the disciplines that produce it.

They form a ladder — crafts, context, harness, loops, orchestration, substrate — and the theme is constant: as models improve, your job climbs from writing prompts to designing the systems that run them.



The Four Crafts

Prompting has not vanished; it has split into four skills — intent, spec, context, and prompt — and the useful question is who owns each ([Ahuja, 2026](#)).

Craft	Owner	What it produces
Intent	Human	The goal, constraints, failure conditions
Spec / expectations	Human	The evaluable definition of done
Context	Harness	The tokens the model sees at each step
Prompt	Harness	The reusable interaction patterns (plays)

Clear ownership is what stops drift, and it has a sharp security edge: compartment the evaluations so the builder cannot see the tests it will be judged on, or it will optimise for the checks instead of the outcome — the reward-hacking failure that a systematic survey of RLHF traces to optimising any imperfect proxy hard enough ([Casper et al., TMLR](#)). The harder discipline is presence. It is tempting to step out and sign off only at the end, but a drifted result is worse than no result because it lies with confidence ([Ahuja, 2026](#)); staying in the loop while the work runs beats approving it at the final gate ([Ahuja, 2026](#)). Presence is also calibration, not just supervision: when people receive real-time, ground-truth feedback as an agent works, their own confidence stops drifting to match the model's — the one intervention shown to break that pull ([Li et al. 2025](#)).

Context Engineering

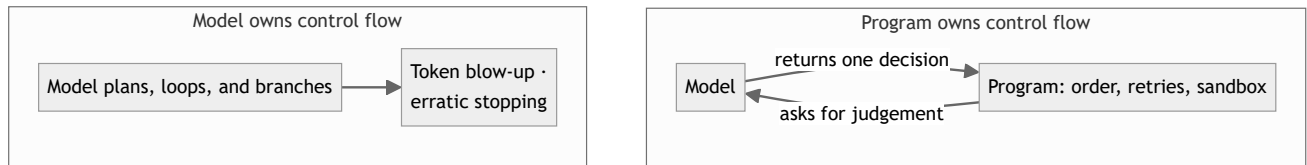
Anthropic calls context engineering the successor to prompt engineering: less about finding the right words, more about curating the tokens an agent sees at each step ([Anthropic 2025](#)). It matters because attention is finite. Chroma's "context rot" study of 18 models shows recall decays as the window fills, and the reassuring needle-in-a-haystack test only measures lexical lookup; once a needle needs a semantic inference, accuracy falls further with length — every token spends an attention budget, a cost of the transformer's n^2 scaling ([Chroma 2025](#)). Context is a scarce resource, not a dumping ground; aim for the smallest set of high-signal tokens that does the job.

That yields concrete techniques. Compaction summarises a near-full conversation and reopens a fresh window with the essentials. Note-taking lets an agent keep a NOTES.md and read it back after a reset. Sub-agents explore on clean contexts and return distilled summaries. Just-in-time retrieval keeps lightweight references — file paths, queries — and loads detail only when needed, so naming and folders become signal. Structure can be injected too: inline call/inheritance tags give a grep-first agent +2.2pp localisation, shorter trajectories, and roughly half the variance — helping less by making agents smarter than by making navigation reproducible ([2606.26979](#)). The recurring waste is re-reading the whole store every turn; the discipline is to feed context progressively.

Harness Engineering

A harness is the runtime around the model: tool use, planning, retries, and sandboxes (isolated environments where generated code can run without touching the real system). The reliable design is to let the program own control flow — the order in which steps run and branch — and call the model

only for judgement, so that runaway token use and erratic stopping stop being mysteries and become engineering ([2606.15874](#)). Decomposing tasks at runtime, so only the failed step reruns rather than the whole pipeline, cuts retry cost by half or more in measured workloads ([2605.15425](#)). The fragile alternative is handing all the looping and branching to a probabilistic system and hoping a better prompt rescues it.



Loopcraft

The craft of the moment is stacking self-correcting cycles and watching their trajectories, since non-deterministic loops break ordinary unit tests and leverage comes from loops, not prompts. The practitioners are blunt about it — Steinberger says design the loops that prompt your agents; Cherny says he writes loops, not prompts; Karpathy says arrange things so they run autonomously and hit go ([Loopcraft](#)). The skill is knowing when to drop a loop for reliability and when to climb one for leverage. The trap is fixing things by hand as you always have, instead of building systems that scale with more agents.

Meta-Harnesses & Orchestration

Above the harness sit harnesses that orchestrate other harnesses — coordinating agents, selecting models, enforcing governance. Leverage now scales with agents rather than your own speed, which makes standard primitives for identity, memory, and orchestration worth real investment. The danger is multi-agent sprawl with no shared identity and no audit trail, where the leverage you bought turns into liability you did not.

The Substrate Stack & Memory

It helps to locate yourself on a ladder of maturity, where each rung is a genuine technological bet rather than a slogan ([Ahuja, 2026](#)):

Level	Substrate	Reality
Vibe	Model + IDE	Fine solo; fragile at scale
Spec	Structured tooling	Step two; collapses on big systems
Intent	Plays + memory + crafts	Where serious tools head
Autonomous	Shared guardrails	Theory for most
Dark factory	Self-running pipeline	Aspirational

Memory is the prerequisite for the upper rungs, because continuity needs an empirical record to reason over, and most teams are honestly nearer the middle than they admit. Name your level before committing to the next; claiming a rung whose substrate you have not built is how certainty gets sold that no one has earned.

Chapter 5 — Responsibility & Governance (the duty)

The first time I gave an agent my own login so it could "just get things done," it worked beautifully — until I realised every action it took wore my name, with no way to tell mine from its. Capability without responsibility is how organisations get hurt, and 2026 supplied the cautionary tales.

This chapter is the duty half of the discipline: securing agents, owning what they say, and governing access in a world where the model you depend on may be pulled or repriced overnight. It is the 愛 in the method made concrete — care expressed as guardrails.

Safety & Red-Teaming

Agent security is not cybersecurity with AI bolted on; it is a new attack surface — agent sessions, browser-extension takeovers, prompt-data exfiltration — that older controls never anticipated. The trend is unflattering: documented AI incidents rose from 233 to 362 in a year, and responsible-AI reporting still trails capability reporting, so the gap between what models do and what we measure widens ([HAI 2026](#)).

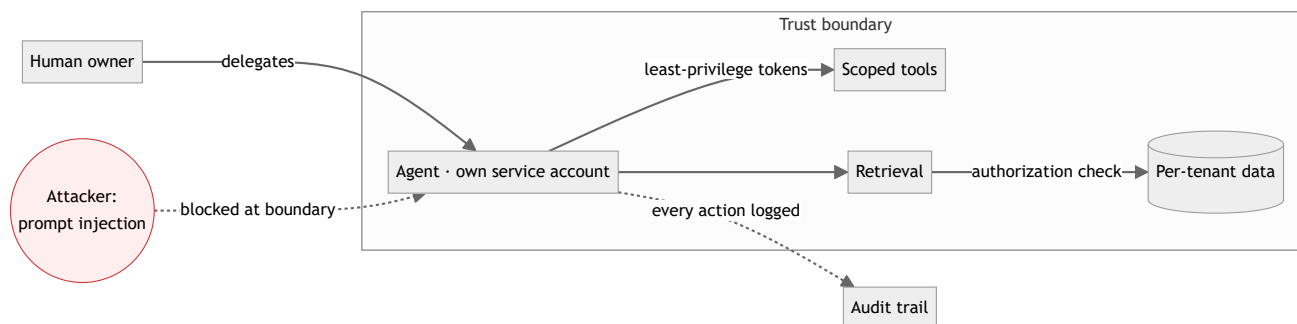
The response is structural: least-privilege access per agent, deliberate red-teaming of sessions, and control loops that monitor an agent's own decisions ([2512.21354](#)). Those loops can in principle watch the model's internal state, not only its outputs: Anthropic's interpretability team found that internal representations of "desperation" causally raise the rate of agentic misalignment — blackmail, and reward-hacking under pressure — while "calm" suppresses it, and they propose monitoring such activations as a runtime warning sign ([Sofroniew et al., Anthropic](#)). In multitenant settings it is sharper still — retrieval ranks by relevance, not authorization, so ungated RAG leaks cross-tenant data in 98–100% of probes. Gate at retrieval, enforce server-side, never trust the client ([2605.05287](#)).

NOTE

Security terms used here, in plain English:

- **RAG (retrieval-augmented generation)** — fetching relevant documents from a store and feeding them to the model as context, so its answer is grounded in your data rather than its training alone.
- **Multitenant / cross-tenant** — one system serving many customers (tenants); a cross-tenant leak is one customer's query pulling back another's data.
- **Red-teaming** — deliberately attacking your own system, under rules, to find weaknesses before a real attacker does.

[!IMPORTANT] Give each agent its own **service-account identity** with **least-privilege** tokens scoped per tool, not a human's broad credentials. Impersonation grants capability without protection and erases the audit trail.



The OWASP Top 10 for LLM applications names the surface concretely, and each entry maps to a control:

Risk	Control
Prompt injection	Segregate system/user input; gate untrusted content
Insecure output handling	Validate/escape before downstream use
Excessive agency	Least-privilege, scoped tools, human checkpoints
Sensitive-info disclosure	Redaction; per-tenant isolation
Overreliance	Verification + reviewer judgement in the loop

Source: [OWASP LLM Top 10](#). Red-teaming under documented rules of engagement turns these from a checklist into a practice.

Liability & Provenance

Once a model speaks for your organisation, its output is your first-party statement, and errors or infringements land on you, not the vendor — a principle courts have begun to enforce. That turns governance into a contract problem as much as a technical one: indemnity, provenance, and transparency belong in every agreement and every pipeline, so you can show where a claim came from and who is answerable for it. Provenance has a standard — C2PA content credentials cryptographically bind origin and edit history to media ([C2PA](#)) — so adopt it rather than improvising. Provenance matters precisely because audiences cannot supply it themselves: in a 606-reader study, people rated AI- and human-written text as equally credible — and the AI version as *clearer and more engaging* — so the burden of disclosure falls on the publisher, not the reader ([Huschens et al. 2023](#)). Treating AI text as someone else's problem is how the liability arrives unannounced.

Overreliance & Convergence

Overreliance is the quiet entry on every risk register, and at the scale of a whole organisation it takes a particular shape. Frontier models do not merely sound confident; they converge. Asked for strategy across many business contexts, they cluster on the same fashionable answers — in one study choosing "differentiate" over "compete on cost" 96% of the time, with richer context moving the answer by only 11% and better prompting by just 2% ([Romasanta, Thomas & Levina 2026](#)). The researchers call it *trendstop*: advice that sounds tailored but steers every company toward the same crowded position.

The danger is in how convergence meets confidence. An organisation corrects itself through friction — Sales says compete on cost, Product says differentiate, and the argument surfaces what either side missed. When everyone consults the same models and arrives, confidently, at the same answer, that friction vanishes, and the agreement reads as validation rather than the artefact it is. Nobody asks "are you insane?" because everyone is seeing the same thing ([Johnson Spink, 2026](#)).

Convergence at the moment of decision is not the only way a model steers a view; the tool reshapes opinions as people merely write with it. In a controlled experiment, a writing assistant tuned to one side of a contested question moved both what participants wrote and the attitudes they reported holding afterwards — a quiet, scalable nudge the authors argue must be monitored and engineered rather than left to chance ([Jakesch et al. 2023](#)). For governance that means treating the opinions built into a vendor's model as a managed dependency, with the same scrutiny you would give any other input to a decision.

The governance response is to protect divergence deliberately. Reserve genuinely consequential decisions for human reasoning before any chat window is opened; treat agreement among AI-assisted analyses as a weaker signal, not a stronger one; and keep a multi-model bench so that, at the least, the models differ. Convergence is cheap to buy and expensive to discover.

The Deferred Ledger

The cheapest way to mismanage AI is to read today's price as the real one. Building software has fallen close to free, so teams now build because they can, not because a need cleared any bar — and the bill is deferred, not escaped. Per-token inference is genuinely cheap and getting cheaper; the exposure is that the all-in economics are capital-funded and negative. OpenAI reportedly lost around five billion dollars in 2024 on roughly a ten-per-cent gross margin, and its own chief executive said even the two-hundred-dollar tier loses money because "people use it much more than we expected." Capital-funded prices do not hold still: in mid-2025 Cursor quietly turned a flat plan into metered credits because newer models spent more tokens per request than the price could carry (Ahuja, 2026). The question for a leader is not what this costs today, but what the organisation will have become by the time it costs what it truly costs.

Kapil Viren Ahuja names three debts that accrue while the meter is cheap and come due on enterprises, not hobbyists (Ahuja, 2026):

Debt	What accrues	When it comes due
Skill	Judgement that is never exercised atrophies	The quarter a hard build-or-don't-build call finally matters
Dependence	Workflows assume generation is free and reliable	When the tool degrades or reprices under you
Carry	Software built without need becomes inventory	Maintained, secured, and repriced for its whole life

Dependence debt is the easiest to miss, because degradation is invisible: Anthropic's own September 2025 postmortem admitted that for about five weeks roughly 30% of Claude Code users received at least one degraded response, and most never knew the instrument was quietly wrong (Ahuja, 2026). The governance answer is not to build less but to restore the brake that cheap building removed. On every initiative, name the person whose job is to ask three questions — who needs this and what breaks for them if it never exists; would we still build it if it cost a week of engineering rather than an afternoon of tokens; and who owns saying no — and make that same person supply the intent. A decision with no owner is where the value question quietly disappears (Ahuja, 2026).

Governed Access

Governance has to cover access, not just usage. Deloitte finds nearly seven in ten organisations running autonomous agents while barely a fifth have mature governance for them, and country-of-origin is now a deciding factor in vendor choice as sovereign-AI concerns grow (Deloitte 2026). A common spine for the work is NIST's AI Risk Management Framework, which organises it into four functions so risk is designed for rather than discovered (NIST AI RMF):

Function	Question it answers
Govern	Who is accountable, and under what policy?
Map	Where could this system cause harm?
Measure	How do we quantify those risks?
Manage	How do we mitigate and monitor them?

The useful frame is to govern the repository rather than the agent — controlling the ecosystem the work lands in, where risk is measurable, instead of policing each model ([2606.28235](#)) — and to keep a multi-model fallback, including open weights, ready.

The config layer that steers agents is itself unmanaged supply chain: a study of 10,008 repos found 10% of agent-config paths are exact duplicates across orgs and under 1% declare permission boundaries. Governance must be deterministic and tool-agnostic, not delegated to more LLM orchestration ([2606.26924](#)). Single-vendor dependence under geopolitical release risk is the exposure to retire first — the cheapest outage is the one you planned for.

Chapter 6 — Mastery & Forward Practice

My skills are more in demand now than ever, and I am turning work away — not despite AI, but because of it. A book ends where a 道 should: not at a destination but at a practice you keep. With models converging and tools commoditising, the lasting questions are what stays human and how to keep improving. This chapter answers both — the edge you protect, and the loop you never stop running.

The Human Edge

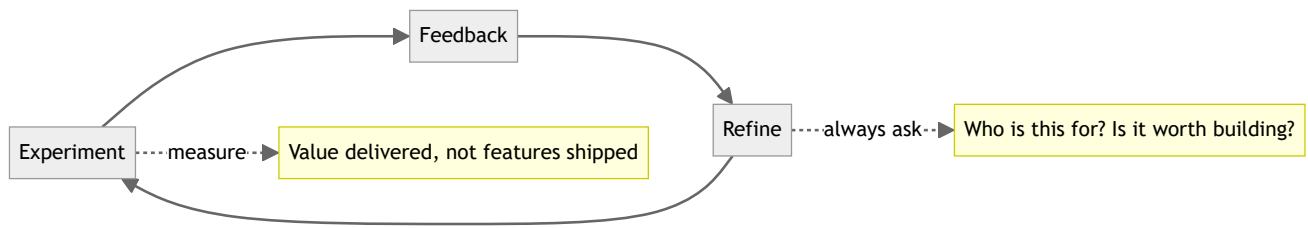
As model quality converges, advantage moves to what cannot be trained: tacit expertise and hands-on practice — the engineer or analyst who sits with the real problem and knows why it matters. The pattern in the data is augmentation, not replacement; firms that over-automated and then rehired seasoned staff make the point in reverse ([HAI 2026](#)). Tellingly, measured gains skew to novices: AI lifts beginners most by encoding what experts already know, which says the durable value is the expertise itself ([Brynjolfsson et al. 2023](#)).

You keep the edge by staying close to real problems and owning intent and judgement. You lose it by hollowing out the junior pipeline that makes tomorrow's seniors — optimising a year that costs a decade. The risk has a measured shape: across 11,097 repositories, agent adoption left human contributor counts flat but cut newcomer share 3.7pp and deepened review effort 5.3% — augmentation with dilution, the entry rungs thinning while maintainer burden climbs ([2606.26289](#)).

There is an inner edge as well as an organisational one: your sense of your own judgement. AI erodes it twice over. It lifts task scores while flattening metacognition, so strong and weak performers end up equally — and wrongly — sure of themselves, and the more someone knows about AI the *worse* their self-assessment tends to become ([Fernandes et al. 2026](#)). And a person's confidence drifts to match whatever confidence the model projects, a pull that lingers even after the model is gone ([Li et al. 2025](#)). Protecting the human edge therefore means protecting calibration: forming a view of your own before you ask, knowing what you actually know, and treating the machine's certainty as one more input to weigh, never a verdict to accept.

Continuous Refinement

The book closes on the idea it opened with: a path. Treat your practice as a loop — experiment, get feedback, refine, repeat — and keep AI human-centred at each turn. The evidence is consistent that the gain comes not from the tool but from redesigning work around it, which is why the high performers move pilots into production while others count demos ([McKinsey 2025](#)).



No method will be the last word, and that is the point of treating practice as a path rather than a destination. Vibe coding was the style of one year and looked spent within six months; spec-driven development began to buckle inside a year; Agile long ago decayed into a spec process with shorter cycles, and test-first development faded before it. What survives every relabelling is the discipline underneath — staying close to the work, holding the intent, asking what is worth building. So invest in that rather than the framework of the season: build the skills that direct the machine instead of racing to out-type it, and refuse to keep score by tokens burned, a number that flatters motion over value — one developer's single month ran to 603 billion tokens ([Ahuja, 2026](#); [Ahuja, 2026](#)).

Measure value, not output. Features shipped is a flattering number while whether anyone needed them goes quietly unasked. The way of AI turns out to be that one question, asked again and again — who is this for, and is it worth building — until it hardens into habit. That habit, more than any model, is what AI-dō is for.